

Final Project Report

ACCESS FORM



Project Supervisor

Abdullah Qamar

Submitted By

F25PROJECT00BCC

Muhammad Arqam

BC220203921

**Software Projects & Research Section,
Department of Computer Sciences,
Virtual University of Pakistan**

CERTIFICATE

This is to certify that [Muhammad Arqam \(BC220203921\)](#) has worked on and completed their Software Project at Software & Research Projects Section, Department of Computer Sciences, Virtual University of Pakistan in partial fulfillment of the requirement for the degree of BS in Computer Sciences under my guidance and supervision.

In our opinion, it is satisfactory and up to the mark and therefore fulfills the requirements of BS in Computer Sciences.

Supervisor / Internal Examiner

Abdullah Qamar,
Supervisor,
Software Projects & Research Section,
Department of Computer Sciences
Virtual University of Pakistan

(Signature)

External Examiner/Subject Specialist

<<[External Supervisor Name](#)>>

(Signature)

Accepted By:

(For office use)

EXORDIUM

In the name of Allah, the Compassionate, the Merciful.

**Praise be to Allah, Lord of Creation,
The Compassionate, the Merciful,
King of Judgment-day!**

**You alone we worship, and to You alone we pray for help,
Guide us to the straight path**

**The path of those who You have favored,
Not of those who have incurred Your wrath,
Nor of those who have gone astray.**

DEDICATION

Every challenging work needs self-effort as well as the guidance of elders, especially those who are very close to our hearts.

I proudly dedicate this final year project to my beloved **Parents**, who have been my ultimate source of inspiration. Their unconditional love, countless sacrifices, and day-and-night prayers gave me the strength to complete my degree. Whatever I have achieved today is solely because of their endless support and firm belief in my abilities. I also dedicate this to my siblings and friends who stood by me during the stressful times of this project.

Furthermore, I dedicate this work to my honorable supervisor, **Sir Abdullah Qamar**. His continuous guidance, patience, and valuable feedback acted as a beacon of light for me. He always pointed me in the right direction, and without his mentorship, the successful completion of the "**Access Form**" system would not have been possible.

Lastly, I dedicate this project to differently abled individuals and the advocates of digital accessibility. My real motivation behind this project was to create a system that ensures nobody is left behind due to physical or visual barriers. It is my sincere hope that this effort serves as a meaningful step toward a more inclusive digital world.

ACKNOWLEDGEMENT

I would like to express my deepest and most sincere gratitude to my project supervisor, **Sir Abdullah Qamar**. His continuous guidance, insightful feedback, and technical expertise were instrumental in the successful completion of the "Access Form" project. I am truly thankful for his patience and mentorship, which helped me navigate complex development challenges and deeply understand the core principles of digital accessibility.

I also extend my sincere appreciation to the Department of Computer Science at the Virtual University of Pakistan. The academic environment and foundational knowledge provided by the faculty throughout my degree program equipped me with the essential software engineering skills required to build this real-world application.

Furthermore, I am thankful to my friends and peers for their constructive discussions, brainstorming sessions, and moral support, which kept me motivated throughout the development lifecycle of this project.

Finally, I want to express my profound gratitude to my parents and family. Their unwavering emotional support, endless encouragement, and personal sacrifices provided me with the resilience needed during the most demanding and stressful phases of this project. I could not have reached this milestone without their unconditional belief in me

PREFACE

When it was time to select a topic for my final year project, I knew I wanted to build something that solves a genuine, real-world problem. In today's digital era, data collection, forms, and surveys are everywhere. However, while researching, I noticed a significant gap: most of the popular survey platforms are built for the average user, completely ignoring individuals with visual, physical, or cognitive impairments. This realization became the foundation of my project, "**Access Form**".

"**Access Form**" is not just another standard survey builder; it is a platform built with absolute inclusivity at its core. My goal was to create a system where web accessibility is not just an afterthought, but the main feature. By integrating tools like smart **voice-to-text typing**, a **built-in screen reader**, **high-contrast** themes, and a **color-blind** safe mode, I wanted to ensure that anyone, regardless of their physical limitations, can comfortably create and participate in surveys.

This project report documents my entire journey of bringing this idea to life. It is structured into four main chapters. **Chapter 1** covers the initial requirement gathering, scope, and use-case scenarios. **Chapter 2** details the project planning and the methodology I chose to follow. **Chapter 3** visually explains the system's architecture through various UML and database diagrams. Finally, **Chapter 4** focuses on the actual development, coding, and implementation phases of the application.

Working on this project has been an incredible learning experience. It not only strengthened my technical skills in web development and software engineering but also taught me the critical importance of **Web Content Accessibility Guidelines (WCAG)**. I hope this report clearly reflects the hard work, passion, and technical effort I have put into developing this inclusive system.

Table of Content

CHAPTER 1

GATHERING & ANALYZING INFO..... 11

1.1.	INTRODUCTION.....	12
1.2.	PURPOSE	12
1.3.	SCOPE.....	12
1.3.1.	Admin Module:	13
1.3.2.	Form Creator Module:	13
1.3.3.	Respondent Module (Public Access):.....	13
1.4.	DEFINITIONS, ACRONYMS AND ABBREVIATIONS.....	14
1.4.1.	WCAG (Web Content Accessibility Guidelines).....	14
1.4.2.	A11y (Accessibility)	14
1.4.3.	Web Speech API (TTS & STT).....	14
1.4.4.	Branching Logic	14
1.4.5.	PHP (Hypertext Preprocessor)	14
1.4.6.	PDO (PHP Data Objects).....	15
1.4.7.	HTML (Hypertext Markup Language).....	15
1.4.8.	CSS (Cascading Style Sheets).....	15
1.4.9.	JavaScript (JS)	15
1.4.10.	HTML2PDF.....	15
1.4.11.	CSV (Comma-Separated Values)	15
1.4.12.	High Contrast / Color-Blind Mode.....	15
1.4.13.	MySQL / MariaDB.....	16
1.4.14.	Virtual University of Pakistan (VU).....	16
1.5.	USE CASES AND USAGE SCENARIOS.....	16
1.5.1.	Use Case Diagrams.....	16
1.5.2.	Usage Scenarios	21
1.6.	SUPPLEMENTARY REQUIREMENTS	25
1.6.1.	Usability.....	25

1.6.2. Reliability.....	25
1.6.3. Supportability.....	25
1.6.4. System Requirements	26

CHAPTER 2

PLANNING THE PROJECT 27

2.1. INTRODUCTION.....	28
2.2. METHODOLOGY	28
2.3. AVAILABLE METHODOLOGIES.....	28
2.3.1. Waterfall Model:	28
2.3.2. Agile Methodology:.....	29
2.3.3. Spiral Model:	29
2.3.4. V-Model.....	29
2.4. CHOSEN METHODOLOGY	29
2.5. REASONS FOR CHOSEN METHODOLOGY	31
2.5.1. Fixed and Clear Requirements:	31
2.5.2. Testing Complex Features Early:.....	31
2.5.3. Best for a Single Developer:	31
2.5.4. Managing Time and Deadlines:.....	31
2.5.5. Better Quality and Reliability:	31
2.6. WORK PLAN	32
2.6.1. Planning and Requirement Gathering	32
2.6.2. System and Database Design	32
2.6.3. Development and Coding	32
2.6.4. Testing and Bug Fixing.....	32
2.7. PROJECT STRUCTURE	33
2.7.1. Team Structure	33
2.7.2. Project Schedule (Submission Calendar)	33

CHAPTER 3

DESIGNING THE PROJECT 35

3.1. INTRODUCTION.....	36
------------------------	----

3.2.	PURPOSE	36
3.3.	SCOPE.....	36
3.4.	DEFINITIONS, ACRONYMS AND ABBREVIATIONS.....	37
3.4.1.	UML (Unified Modeling Language)	37
3.4.2.	ERD (Entity Relationship Diagram).....	37
3.4.3.	Sequence Diagram	37
3.4.4.	Class Diagram	37
3.4.5.	PK and FK (Primary Key & Foreign Key)	38
3.4.6.	GUI (Graphical User Interface).....	38
3.4.7.	Multi-Tier Architecture	38
3.5.	DYNAMIC MODEL: SEQUENCE DIAGRAMS	38
3.5.1.	User Authentication (System Wide)	39
3.5.2.	Manage Users (Admin Panel).....	40
3.5.3.	Create New Survey (Creator Panel)	41
3.5.4.	Build Survey / Add Question (Creator Panel).....	42
3.5.5.	Publish Survey & Generate Link	43
3.5.6.	Fill & Submit Survey (Respondent Panel).....	44
3.5.7.	View & Export Responses	45
3.5.8.	Configure Accessibility Settings	46
3.6.	OBJECT MODEL/LOGICAL MODEL: CLASS DIAGRAM	47
3.7.	DATABASE MODEL (DATABASE DIAGRAM)	48
3.8.	GRAPHICAL USER INTERFACES	49
3.8.1.	Authentication Screens:.....	49
3.8.2.	Admin Panel:	51
3.8.3.	Form Creator Panel:	52
3.8.4.	Respondent Panel:	54
3.8.5.	Accessibility Features	55

CHAPTER 4

DEVELOPMENT 57

4.1.	DEVELOPMENT PLAN (ARCHITECTURE DIAGRAM)	58
------	---	----

4.1.1. Presentation Layer (Frontend):.....	58
4.1.2. Application Layer (Backend Logic):	58
4.1.3. Data Layer (Database):.....	58

CHAPTER 1

Gathering & Analyzing Info

1.1. INTRODUCTION

In today's digital world, online forms and surveys have become the primary method for gathering data, feedback, and research. However, a major problem in this domain is that most existing survey platforms are designed for the average user, completely ignoring the needs of individuals with visual, cognitive, or physical impairments. This creates a digital barrier for differently-abled people who want to share their opinions independently. To solve this real-world issue, I have developed the "**Access Form**" system.

"**Access Form**" is a modern, web-based survey and data collection platform built specifically with digital accessibility and **WCAG** (Web Content Accessibility Guidelines) principles in mind. The system accommodates three main types of users: **Administrators**, **Form Creators**, and **Respondents**. It provides a seamless environment where Form Creators can build dynamic surveys with **branching logic**, while Admins monitor the overall system.

What truly makes this project unique is its **built-in accessibility toolkit** for the Respondents. Instead of forcing users to rely on expensive third-party assistive software, "**Access Form**" provides native features like **Smart Voice Typing** (Voice-to-Text), an **automated Screen Reader** (Text-to-Speech), complete **Keyboard Navigation**, High Contrast themes, and a Color-Blind safe mode directly within the website. Through this project, the goal is to demonstrate that web applications can be made fully inclusive, allowing everyone to participate without any digital barriers.

1.2. PURPOSE

The primary purpose of the "**Access Form**" project is to provide a barrier free, inclusive platform for online data collection and surveys. Usually, people with **visual** or **physical impairments** face significant difficulties while interacting with standard web forms because traditional UIs (User Interfaces) are not designed with accessibility in mind.

The core objective of this system is to eliminate these digital barriers by embedding assistive tools such as **voice typing**, **screen readers**, and **high contrast** modes directly into the web application. By doing this, the system empowers differently abled users to independently read, navigate, and fill out surveys without needing any external or paid accessibility software. Furthermore, it serves the purpose of helping organizations, researchers, and educators collect more diverse and accurate data, ensuring that no demographic is excluded from sharing their valuable feedback due to inaccessible technology.

1.3. SCOPE

The scope of the "**Access Form**" project is defined by the boundaries of a comprehensive, web-based survey management system. It does not focus on e-commerce or complex third-party API integrations, but rather strictly targets the creation of an inclusive, highly accessible data collection environment. The entire system is divided into three detailed modules, each catering to a specific user role with its own distinct boundaries and functionalities:

1.3.1. Admin Module:

The **Administrator** acts as the overarching controller of the platform. The scope of this module includes system wide monitoring and user management. The Admin has the authority to view all **registered users**, check their **disability profiles**, and enable, disable, or permanently delete accounts to maintain system security. Additionally, the Admin can monitor all surveys created across the platform, view total response counts, and export overall system data in offline formats like **CSV (Excel)** and **PDF**.

1.3.2. Form Creator Module:

This module is the core **survey building** engine. Registered Form Creators have a dedicated dashboard where they can create, draft, and publish surveys. The scope here includes adding various types of questions (**Text, Multiple Choice, Rating, Boolean**) and organizing them using Section Headers. A major feature within this scope is "**Branching Logic**," which allows creators to set conditional questions that only appear based on a respondent's previous answers. Once a survey is published, the creator can generate a public link to share, view real-time responses, and export their specific survey data for analysis.

1.3.3. Respondent Module (Public Access):

This module defines the experience for the end users filling out the survey. Respondents access the survey via a **shared public link**. Upon entering, they are prompted to select a specific "**Disability Profile**" (e.g., Visual Impairment, Color Blindness, Physical Impairment), which automatically tailors the system's UI to their needs. The scope of this module heavily emphasizes accessibility tools, including:

- **Smart Voice Typing:** Allowing users with motor disabilities to answer questions using speech-to-text (activated via Alt+M).
- **Integrated Screen Reader:** Automatically reading out questions, options, and page titles for visually impaired users.
- **Visual Enhancements:** Offering High Contrast (Dark Mode) and a Color-Blind safe palette to accommodate users with specific vision deficiencies.
- **Keyboard Navigation:** Ensuring the entire survey can be navigated securely using only the Tab and Arrow keys.

By keeping the scope bounded to these specific modules and accessibility tools, the project delivers a highly focused, practical, and real-world solution for inclusive data collection without overcomplicating the system with unnecessary features.

1.4. DEFINITIONS, ACRONYMS AND ABBREVIATIONS

To ensure that this project report is easy to understand, the following key terms, technologies, and abbreviations have been explained along with their specific role in this system:

1.4.1. WCAG (Web Content Accessibility Guidelines)

- **Definition:** A set of internationally recognized guidelines created by the W3C (World Wide Web Consortium) that explain how to make web content accessible to people with disabilities.
- **Relevance in Project:** This is the core standard upon which the entire "Access Form" UI/UX is designed.

1.4.2. A11y (Accessibility)

- **Definition:** A common alphanumeric abbreviation for the word "Accessibility" (there are 11 letters between the starting 'A' and the ending 'y').
- **Relevance in Project:** Used as a standard naming convention in the project's codebase (e.g., accessibility.js, a11y-controls CSS classes) for all accessibility-related scripts.

1.4.3. Web Speech API (TTS & STT)

- **Definition:** A native web browser API that provides Text-to-Speech (TTS) and Speech-to-Text (STT) capabilities without needing third-party software.
- **Relevance in Project:** Acts as the primary engine for the system's "Smart Voice Typing" (user speaks to type) and "Built-in Screen Reader" (system reads questions aloud).

1.4.4. Branching Logic

- **Definition:** A conditional rule system in surveys where the display of a subsequent question depends entirely on the answer provided to a previous question.
- **Relevance in Project:** Allows Form Creators to build smart, dynamic surveys where irrelevant questions are hidden from respondents based on their real-time answers.

1.4.5. PHP (Hypertext Preprocessor)

- **Definition:** A widely-used open-source server-side scripting language especially suited for web development.
- **Relevance in Project:** Serves as the core backend language of the system, handling routing, form submissions, session management, and server side business logic.

1.4.6. PDO (PHP Data Objects)

- **Definition:** A secure database access layer providing a uniform method of access to multiple databases in PHP.
- **Relevance in Project:** Used to securely connect to the MySQL database. It prevents SQL Injection attacks by utilizing prepared statements for all user and survey queries.

1.4.7. HTML (Hypertext Markup Language)

- **Definition:** The standard markup language used to create the structure and skeleton of web pages.
- **Relevance in Project:** Used to build the fundamental layout of all system screens, including the Admin dashboard, Form Builder, and the public survey forms.

1.4.8. CSS (Cascading Style Sheets)

- **Definition:** A styling language used to control the visual presentation, layout, and colors of web pages.
- **Relevance in Project:** Highly critical in this project to implement the accessibility themes. CSS is used to toggle the "High Contrast" (Dark Mode) and "Color-Blind Safe" layouts dynamically.

1.4.9. JavaScript (JS)

- **Definition:** A programming Language that adds interactivity and dynamic behavior to web pages.
- **Relevance in Project:** The Backbone of the frontend logic. It handles the dynamic branching logic of questions, triggers the accessibility features instantly without page reloads, and processes the voice commands.

1.4.10. HTML2PDF

- **Definition:** A JavaScript library that converts standard HTML web pages into fully formatted PDF documents entirely on the client side.
- **Relevance in Project:** Used in the Admin and Creator panels to allow users to generate and download offline PDF reports of survey responses with proper tables and margins.

1.4.11. CSV (Comma-Separated Values)

- **Definition:** A plain-text file format that uses a comma to separate values, widely used for data exchange between applications.
- **Relevance in Project:** Used to export survey data so that Form Creators can open and analyze respondent answers in spreadsheet software like Microsoft Excel.

1.4.12. High Contrast / Color-Blind Mode

- **Definition:** Specific visual UI adjustments that alter the color palette of a website to make text and elements readable for visually impaired users.

- **Relevance in Project:** The system uses JavaScript to toggle these modes dynamically, converting confusing colors (like standard red/green) into universally readable combinations (like magenta/yellow or deep contrast).

1.4.13. MySQL / MariaDB

- **Definition:** A widely used, open-source relational database management system (RDBMS) that stores data in structured tables.
- **Relevance in Project:** Serves as the central repository for the system, storing user credentials, accessibility preferences, dynamic survey questions, and respondent answers.

1.4.14. Virtual University of Pakistan (VU)

- **Definition:** The academic institute under which this final year software project is being developed and submitted.

1.5. USE CASES AND USAGE SCENARIOS

1.5.1. Use Case Diagrams

The Use Case Diagrams represent the interaction between users and the system visually. These diagrams show how Admin, Form Creator, and Respondent perform different tasks in the system.

The Admin can manage users, monitor surveys, and generate reports. The Form Creator can create surveys, add questions, apply accessibility settings, and analyze responses. The Respondent can fill and submit surveys using accessible features like keyboard navigation and voice input.

These diagrams help in understanding the overall functionality of the system in a simple and clear way.

➤ **Entire System**

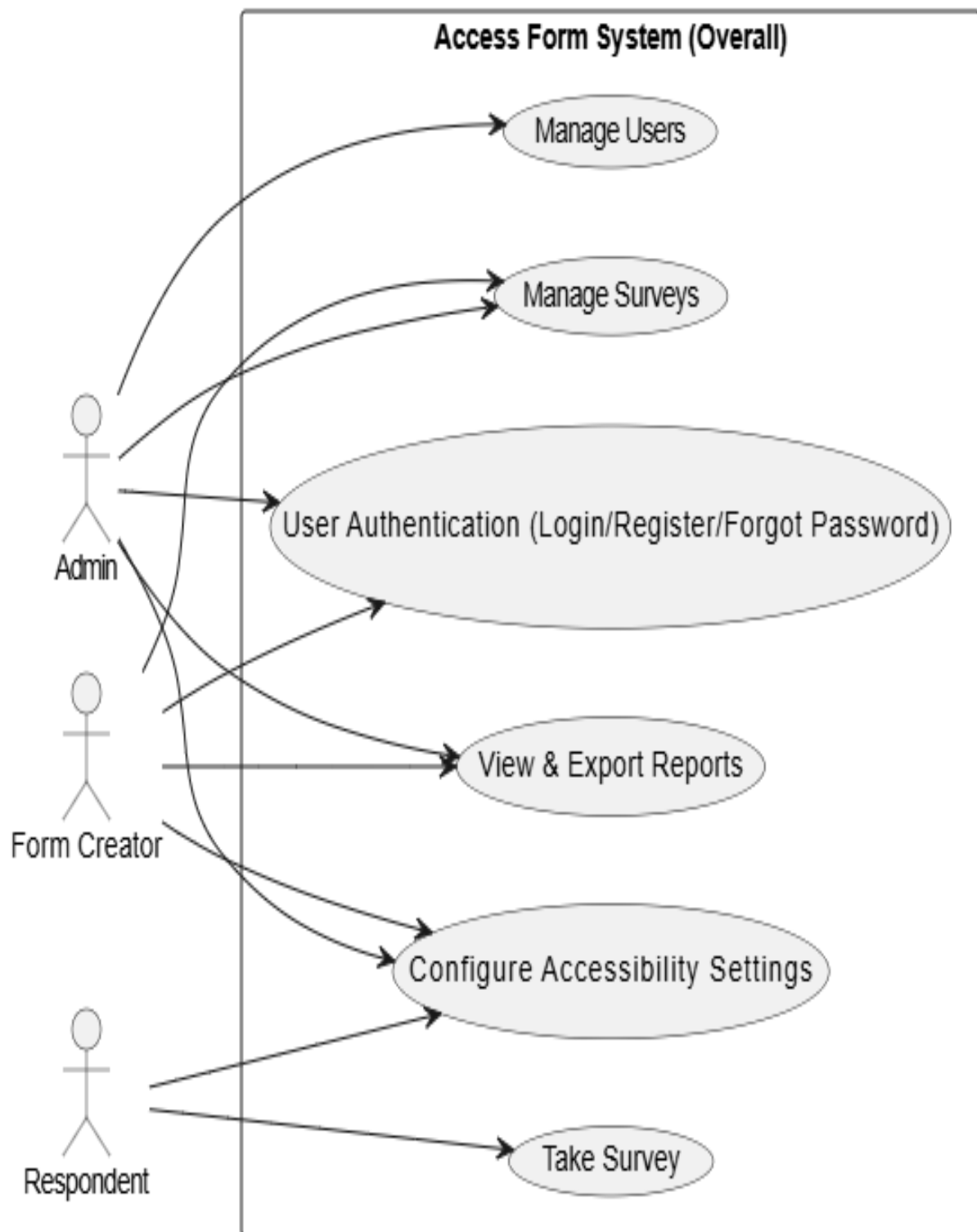


Figure 1.1 - USE CASE DIAGRAM - ACCESS FORM SYSTEM

➤ **Admin Panel**

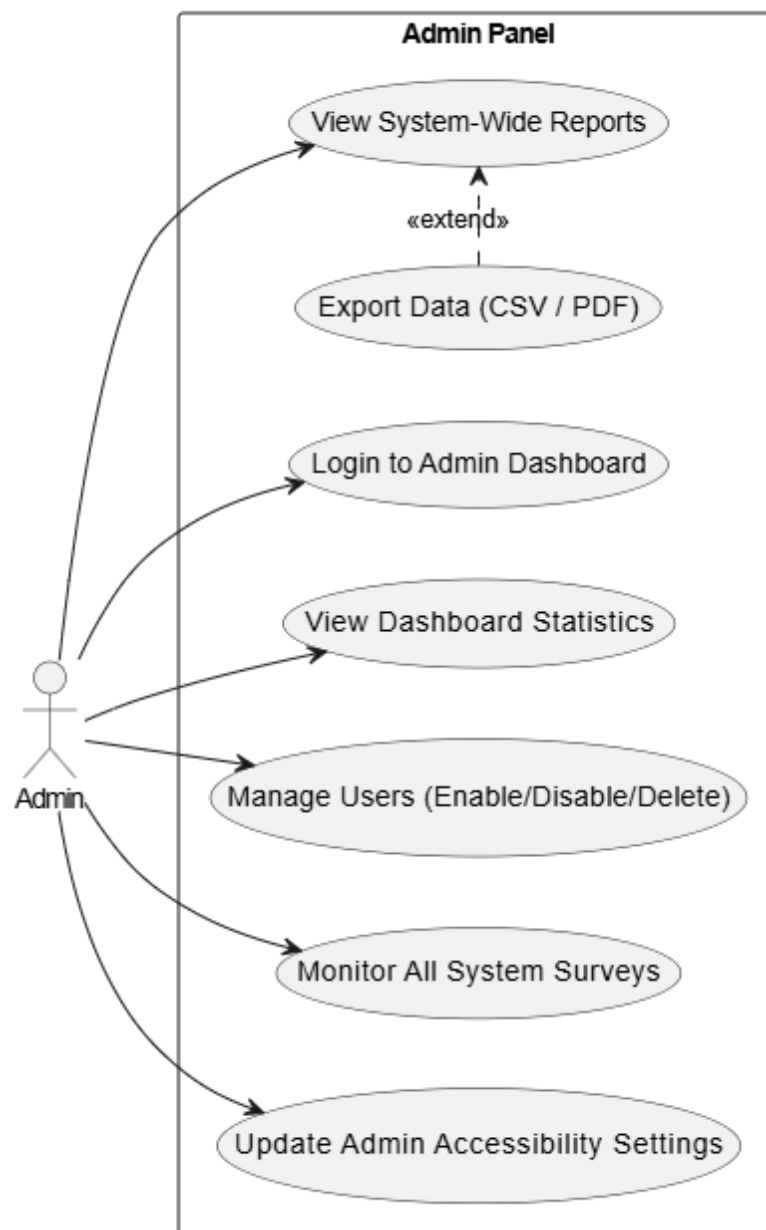


Figure 1.2 - USE CASE DIAGRAM - ADMIN PANEL

➤ **Creator Panel**

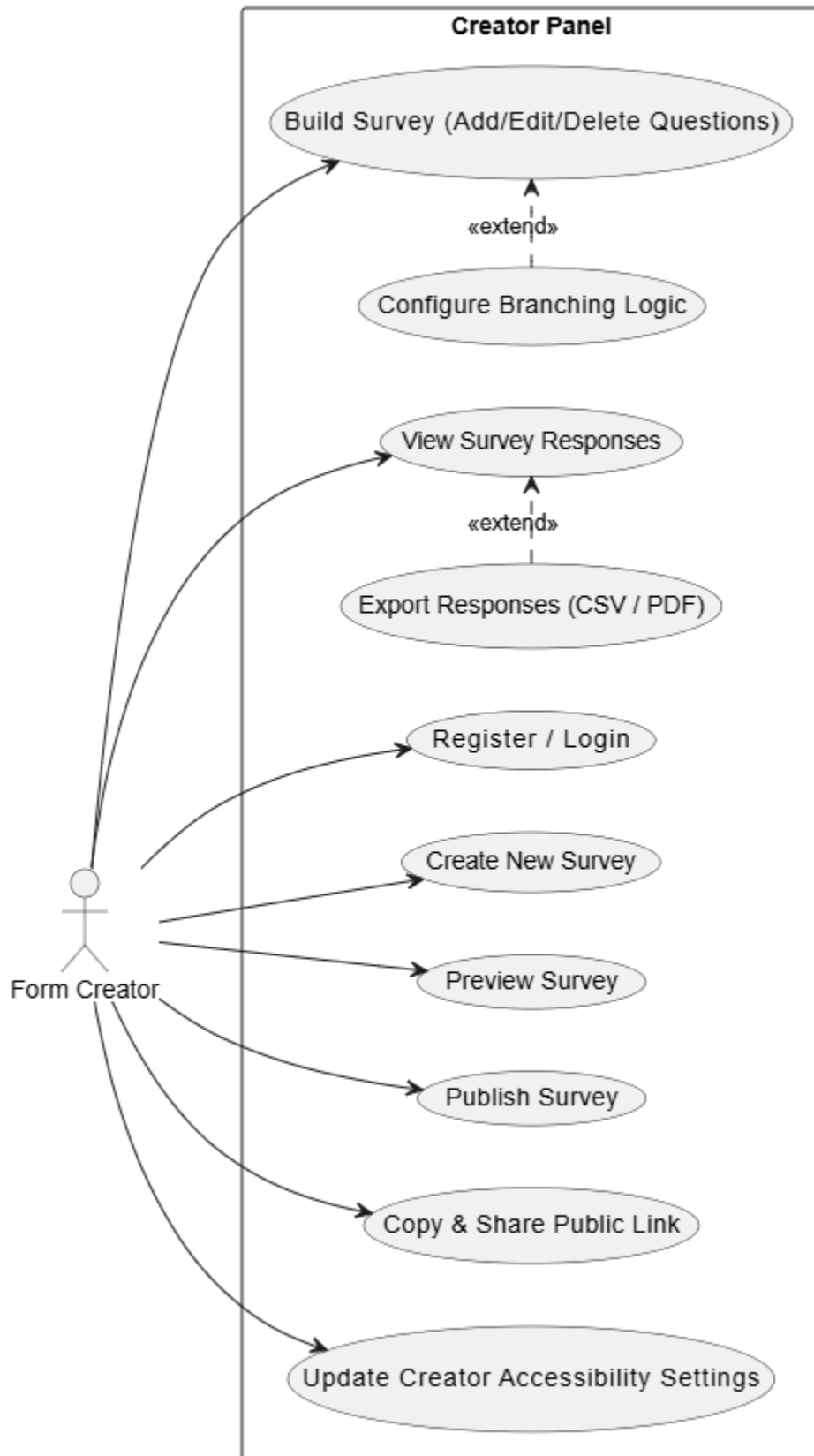


Figure 1.3 - USE CASE DIAGRAM - FORM CREATOR PANEL

➤ **Respondent Panel**

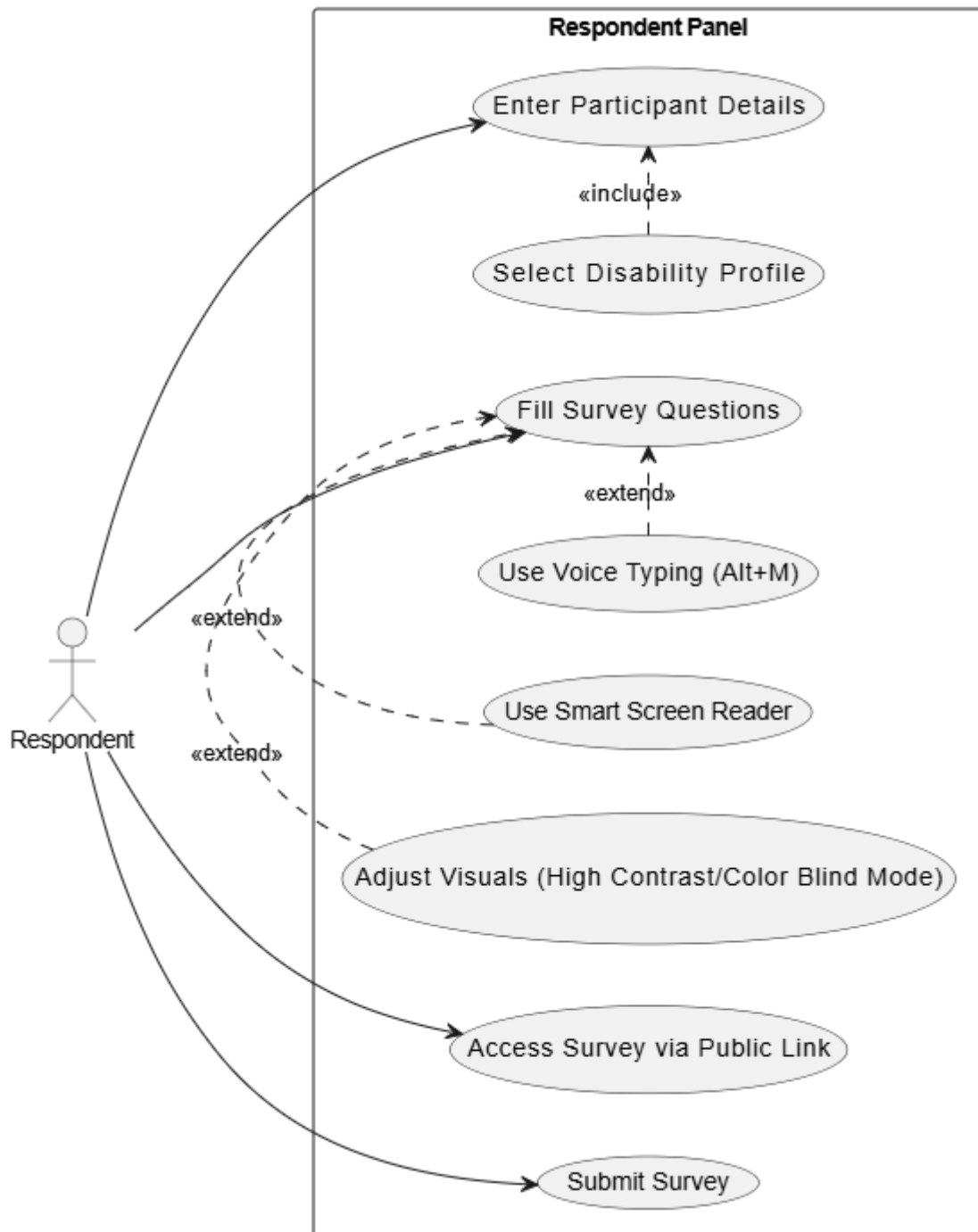


Figure 1.4 - USE CASE DIAGRAM - RESPONDENT PANEL

1.5.2. Usage Scenarios

Table 1 - MANAGE USERS (ADMIN PANEL)

Heading	Description
Use Case Title	Manage Users
Use Case ID	UC-AP-02
Actions	• Admin Logs into the system • Open the admin panel • Select "Users" from the sidebar navigation • View the list of all registered users and their disability profiles • Click "Enable/Disable" to suspend a user, or "Delete" to permanently remove them
Description	Admin manages system users, checks their roles, and can restrict access or delete accounts if needed.
Alternative Paths	• If Admin clicks "Delete" by mistake, they can click "Cancel" on the confirmation popup. • If the user list is very long, Admin scrolls through the table to find specific users.
Pre-Conditions	• Admin must be logged in with Admin privileges. • Users must exist in the database.
Post-Conditions	Admin successfully updates the user's status or removes them from the system.
Author	BC220203921
Exceptions	• Database connection error during status update. • Trying to delete a user who has already been removed.

Table 2 - CREATE NEW SURVEY (CREATOR PANEL)

Heading	Description
Use Case Title	Create New Survey
Use Case ID	UC-CP-01
Actions	• Form Creator logs into the system • Navigate to "Create Survey" menu • Enter Survey Title, Description, and select a Category • Click the "Create Survey" button • System saves the survey as a "Draft" and redirects to the Form Builder
Description	Form Creator initializes a new survey draft before adding any specific questions to it.
Alternative Paths	If the creator leaves the description empty, the system accepts it as it is an optional field.
Pre-Conditions	• User must be logged in as a "Form Creator".
Post-Conditions	A new survey is created with "Draft" status, ready for questions to be added.
Author	BC220203921
Exceptions	• Mandatory field (Title) left blank; system throws validation error. • Server error prevents saving the survey.

Table 3 - BUILD SURVEY / ADD QUESTION (CREATOR PANEL)

Heading	Description
Use Case Title	Build Survey (Add Questions & Branching)
Use Case ID	UC-CP-02
Actions	- Form Creator opens a drafted survey in the "Form Builder" - Select question type (Text, Multiple Choice, Boolean, etc.) - Enter question text and accessibility Alt-Text - Set Branching Logic if required (e.g., Show this ONLY IF Q1 = "Yes") - Click "Add Item" to save the question
Description	Creator adds interactive questions, sections, and smart branching logic to their drafted survey.
Alternative Paths	Creator clicks "Edit" or "Delete" on an already existing question to modify the survey structure.
Pre-Conditions	- Creator must be logged in. - Survey must be in "Draft" status (not published yet).
Post-Conditions	The question and its branching logic are successfully added to the survey database.
Author	BC220203921
Exceptions	- Invalid branching logic selected. - System fails to load existing questions list due to network error.

Table 4 - FILL & SUBMIT SURVEY (RESPONDENT PANEL)

Heading	Description
Use Case Title	Fill and Submit Survey
Use Case ID	UC-RP-01
Actions	- Respondent opens the public survey link - Enters Name, Email, and Disability Profile - System auto-configures accessibility features - Respondent fills out questions using mouse, keyboard, or voice - Clicks "Submit Survey" on the final page
Description	Respondent securely accesses the survey, provides answers using accessible tools, and submits the response.
Alternative Paths	If a respondent misses a required question, the system highlights the missing field and prevents submission.
Pre-Conditions	- Survey status must be "Published". - Respondent has a valid public link.
Post-Conditions	The respondent's answers are saved in the database, and a Thank You screen is shown.
Author	BC220203921
Exceptions	- Respondent uses an email that has already submitted this survey. - Internet disconnects during submission.

Table 5 - CONFIGURATION ACCESSIBILITY SETTINGS

Heading	Description
Use Case Title	Configure Accessibility Settings
Use Case ID	UC-SYS-01
Actions	• User clicks on the Accessibility Menu at the top header • Selects options like A+, A-, High Contrast, Color Blind Palette, or Screen Reader • System instantly applies the visual/audio changes to the UI • System saves these preferences in the database
Description	Any user (Admin, Creator, or Respondent) customizes the UI to accommodate their specific physical or visual disability.
Alternative Paths	User selects a specific "Disability Profile" during registration, and the system automatically enables the required settings.
Pre-Conditions	• The system interface must be loaded completely.
Post-Conditions	Interface adjusts colors, font size, and voice settings according to the user's preferences.
Author	BC220203921
Exceptions	• Browser settings block the Screen Reader Speech API. • Local Storage is restricted by the user's browser, preventing settings from being saved.

Table 6 - USER AUTHENTICATION (SYSTEM WIDE)

Heading	Description
Use Case Title	User Authentication
Use Case ID	UC-SYS-02
Actions	• User enters Email and Password on the Login page • System verifies credentials from the database • System checks if the user account is active • System identifies the role (Admin, Form Creator, or Respondent) • User is redirected to their specific Dashboard
Description	Allows registered users to securely access their respective panels based on their assigned roles.
Alternative Paths	• User clicks "Forgot Password" to receive a reset link if they don't remember their password. • User clicks "Register here" if they don't have an account.
Pre-Conditions	• User must be registered in the system.
Post-Conditions	User is successfully logged in and their session is created.
Author	BC220203921
Exceptions	• Enter Incorrect email or password • Account disabled by Admin

Table 7 - PUBLISH SURVEY & GENERATE LINK

Heading	Description
Use Case Title	Publish Survey & Share Link
Use Case ID	UC-CP-03
Actions	- Form Creator opens a drafted survey in the Form Builder - Clicks the "Publish" button - System checks if the survey has at least one question - System changes survey status from "Draft" to "Published" - Creator clicks "Copy Link" to generate a public URL for respondents
Description	Creator finalizes the survey and makes it live so that respondents can start filling it out via a public link.
Alternative Paths	Creator chooses to "Preview" the survey one last time before clicking Publish.
Pre-Conditions	- Survey must be in "Draft" status. - Survey must contain at least one question.
Post-Conditions	Survey becomes live ("Published") and is locked for further editing.
Author	BC220203921
Exceptions	Creator tries to publish an empty survey

Table 8 - VIEW & EXPORT RESPONSES

Heading	Description
Use Case Title	View & Export Responses
Use Case ID	UC-CP-04
Actions	- Creator/Admin navigates to the "Responses" or "Reports" tab - Selects a published survey to view its data - System displays a tabular view of all respondent answers - User clicks the "Export Data" button - Selects either "Export Excel (.xls)" or "Export .PDF" - System generates and downloads the file
Description	Allows the Admin or Form Creator to analyze survey results online and download them for offline reporting.
Alternative Paths	User just views the data on the screen and clicks "Back" without exporting.
Pre-Conditions	Survey must have at least one submitted response.
Post-Conditions	A formatted PDF or CSV/Excel file is downloaded to the user's local device.
Author	BC220203921
Exceptions	- File generation library (html2pdf) fails to load due to a network issue. - User tries to view responses for a survey with 0 submissions.

1.6. SUPPLEMENTARY REQUIREMENTS

Supplementary requirements define the non-functional aspects of the "**Access Form**" system. These include how the system performs, how secure it is, how easy it is to use for differently-abled individuals, and the specific environment required to run it smoothly.

1.6.1. Usability

Usability is the absolute core of this project. The system is built on the philosophy that technology should adapt to the user, not the other way around.

- **Accessibility First:** The user interface follows WCAG 2.1 guidelines. Everything from contrast ratios to button sizes has been thoughtfully designed.
- **Keyboard Navigation:** Users with physical disabilities who cannot use a mouse can comfortably navigate the entire platform using just the "Tab", "Enter", and "Arrow keys". Focus states are clearly highlighted with thick borders.
- **Assistive Inputs & Outputs:** The integration of the Web Speech API ensures that visually impaired users can hear the questions and physically impaired users can answer them using their voice (Speech-to-Text via Alt+M).
- **Disability Profiles:** Instead of forcing users to find hidden settings, respondents are asked about their accessibility needs right at the start. The system then automatically adapts the UI (e.g., triggering Dark Mode or Color-Blind Palette) based on their selection.

1.6.2. Reliability

The system is designed to handle sensitive survey data securely and reliably, ensuring that responses are never lost or compromised.

- **Data Integrity & Security:** All database interactions are written using PHP Data Objects (PDO) with prepared statements, completely neutralizing SQL Injection attacks. Passwords are mathematically hashed using the bcrypt algorithm before being stored.
- **Session Management:** The system securely manages user sessions. If an Admin or Creator logs out, their session is completely destroyed, preventing unauthorized access via browser history.
- **Graceful Degradation:** If a user accesses the survey on an outdated browser that does not support the microphone, the system does not crash; it simply falls back to standard keyboard and mouse inputs seamlessly.

1.6.3. Supportability

Supportability refers to how easily the system can be maintained, updated, and accessed across different platforms.

- **Cross-Browser Compatibility:** The application uses standard HTML5, CSS3, and JavaScript, ensuring it runs smoothly on all modern web browsers like Google Chrome, Mozilla Firefox, and Microsoft Edge.
- **Responsive Design:** The CSS is written using modern Flexbox and Grid techniques, making the UI responsive. Whether an Admin is viewing reports on a

laptop or a Respondent is taking a survey on a tablet, the interface adjusts itself perfectly.

- **Modular Architecture:** The codebase is neatly divided into separate directories (admin, creator, respondent, includes, assets). This modular structure makes it very easy for future developers to maintain, debug, or add new features without breaking existing functionalities.

1.6.4. System Requirements

To deploy and use the "Access Form" system efficiently, the following minimum hardware and software environments are required:

- **Hardware Requirements:**

- 1. Processor:** Intel Core i3 / AMD Ryzen 3 (or equivalent modern processor).
- 2. Memory (RAM):** 4 GB or higher.
- 3. Storage:** 500 MB of free disk space (for local server installation and database storage).
- 4. Peripherals:**
 - Standard Keyboard.
 - Microphone: Strictly required for respondents utilizing the Voice Typing feature.
 - Speakers/Headphones: Strictly required for the Built-in Screen Reader functionality.

- **Software Requirements:**

- 1. Operating System:** Windows 10/11, macOS, or any standard Linux distribution.
- 2. Local Server Environment:** XAMPP, WAMP, or any stack supporting Apache and MySQL.
- 3. Backend Language:** PHP (Version 7.4 or higher recommended).
- 4. Database:** MySQL (Version 5.7+) or MariaDB.
- 5. Web Browser:** A modern, updated web browser. Google Chrome is highly recommended for full compatibility with the Web Speech API used in this project.

CHAPTER 2

Planning the Project

2.1. INTRODUCTION

In any software project, planning is the most important step before we start coding. For the "**Access Form**" system, I knew I had to make a proper roadmap first. Since this project focuses a lot on accessibility features like voice typing and screen readers, jumping straight into coding without a plan would have caused a lot of errors and confusion later on.

This chapter is all about how I planned the entire project. It explains the software development methodology I looked at, the one I finally chose, and how I managed my time. I have also included a Work Plan (Gantt chart) to show how I divided the project into small tasks over the given time. The main goal of this planning phase was to make sure the project is completed on time, works perfectly for disabled users, and meets all the final year project requirements of Virtual University.

2.2. METHODOLOGY

In software engineering, a methodology is basically a structured way to build a software system. You can think of it as a step-by-step formula. If we try to build a project without following any proper methodology, things will get messy very quickly, and we might miss important requirements or face major bugs in the end.

A Software Development Life Cycle (SDLC) methodology guides us through different phases like understanding the requirements, designing the interfaces, writing the code, and finally testing the whole system. For my "Access Form" project, understanding and applying a proper methodology was very important because I had to manage the database, PHP backend, and complex JavaScript accessibility features all by myself. Following a standard methodology kept me on track and helped me complete the project without getting overwhelmed.

2.3. AVAILABLE METHODOLOGIES

During our software engineering courses at VU, we studied several Software Development Life Cycle (SDLC) models. Every model has its own strengths and weaknesses, and choosing the right one depends on the size of the team, the project deadline, and how clear the initial requirements are. Before finalizing the methodology for the "Access Form" project, I researched and considered the following commonly available methodologies:

2.3.1. Waterfall Model:

This is the most traditional and oldest SDLC model. In the Waterfall model, the development process flows downwards in a straight line, like a real waterfall. You have to completely finish one phase (like Requirements) before moving on to the next phase (like Design). I considered this model because it is very easy to understand. However, its biggest drawback is that it is not flexible. If I wanted to change a feature in the middle of development, going back would be very difficult.

2.3.2. Agile Methodology:

Agile is currently the most popular methodology in the software industry. Instead of building the whole project at once, Agile breaks the project into small chunks called "sprints". After every sprint, a small working part of the software is delivered and tested. While Agile is excellent for large teams and projects with constantly changing requirements, I realized it might be a bit too complex for a solo final year project where the requirements are already fixed by the supervisor.

2.3.3. Spiral Model:

The Spiral model is highly focused on risk analysis. The project passes through a spiral multiple times, and each loop represents a phase. While it is great for very large, expensive, and critical projects, it requires a lot of expertise in risk evaluation. For a standard web-based survey system like mine, using the Spiral model would be over-complicating things unnecessarily.

2.3.4. V-Model

The V-Model is an extension of the Waterfall model, but instead of moving in a straight line downwards, the process steps are bent upwards after the coding phase, forming a "V" shape. The best part about the V-Model is that for every development stage, there is a corresponding testing stage planned right alongside it. This means testing is not left for the very end.

2.4. CHOSEN METHODOLOGY

After carefully reviewing the available Software Development Life Cycle models and keeping the specific needs of the "Access Form" project in mind, I have chosen the **V-Model** (Validation and Verification Model) as my development methodology.

The V-Model operates on a very disciplined and strict approach. Unlike the traditional Waterfall model where testing is left entirely for the final stage, the V-Model proves its uniqueness by pairing every development phase with a corresponding testing phase. The process shape looks like the letter "V", where the left side represents the Verification phases, the bottom tip represents the actual Coding, and the right side represents the Validation phases.

I selected this methodology because my project heavily relies on web accessibility features such as the Web Speech API (for voice typing and screen reading) and dynamic UI adjustments (High Contrast and Color-Blind modes). If I had left the testing of these complex JavaScript APIs to the very end of the project, finding and fixing bugs would have been a nightmare. By using the V-Model, I was able to plan my testing strategies right when I was designing the system requirements.

Working of V-Model in my Project:

- 1. Requirements & Acceptance Testing:** When I gathered the initial requirements for the accessibility features, I simultaneously planned how the final user (Admin/Creator/Respondent) would accept and test them.

- 2. System Design & System Testing:** While designing the overall database and system architecture, I planned the system tests to ensure all modules (Admin, Creator, Respondent) would communicate flawlessly.
- 3. Module Design & Unit Testing:** When I designed individual functions (like the PHP script to save survey answers or the JS function to toggle dark mode), I planned unit tests to verify each specific function in isolation.
- 4. Coding:** Finally, at the bottom of the "V", I wrote the actual PHP, HTML, CSS, and JavaScript code. Once the code was written, the process moved upwards on the right side, executing all the tests I had planned during the design phases.

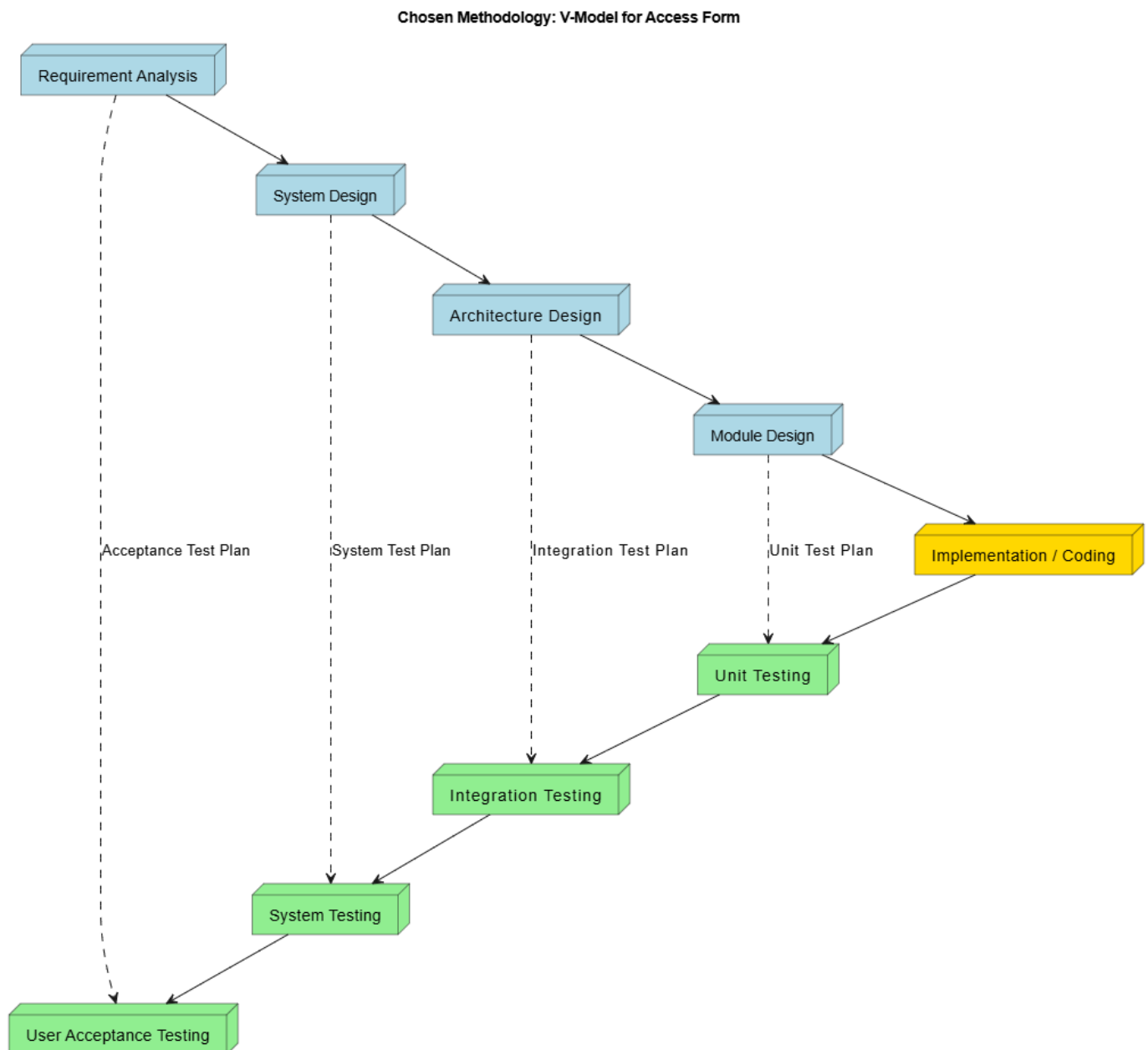


Figure 2.1 - V-Model

2.5. REASONS FOR CHOSEN METHODOLOGY

Selecting the right methodology was a very important step before building the "Access Form" web application. While methodologies like Agile are very famous these days, they are mostly used by big teams where project requirements keep changing every week. Since I am developing this web application entirely by myself and my requirements were already clear from day one, I needed a simple and disciplined approach. That is why I selected the V-Model.

Below are the main reasons why I found the V-Model to be the best fit for developing this web application:

2.5.1. Fixed and Clear Requirements:

Before I even started coding, I knew exactly what features I had to build. I had a clear list of modules: an Admin panel to manage things, a Creator panel to make surveys with branching logic, and a Respondent panel with accessibility features. When the requirements are clear and fixed from the beginning, the V-Model is very easy to follow because you don't have to go back and change things again and again.

2.5.2. Testing Complex Features Early:

My web application uses some tricky JavaScript features, like the Web Speech API for voice typing and screen reading. These features can easily break if the user is on a different browser. If I used a simple Waterfall model and waited until the very end to test these things, fixing bugs would have been a big headache. The V-Model is great because it makes you plan your testing at the same time you are doing your design. This saved me a lot of time later on.

2.5.3. Best for a Single Developer:

I am building the frontend, backend, and database of this system all by myself. Agile methodology requires daily meetings, sprints, and continuous feedback, which is impossible for a single developer. The V-Model gave me a straight, step-by-step path. It told me exactly when to focus on the database design, when to write the PHP code, and when to start testing. It kept me highly focused and organized.

2.5.4. Managing Time and Deadlines:

I had a strict deadline to complete this web application. The V-Model naturally helped me divide my time properly. The left side of the "V" shape took care of my system design, the bottom part was dedicated to coding, and the right side was reserved for testing the application. This clear structure helped me finish the project on time without rushing at the last minute.

2.5.5. Better Quality and Reliability:

Because this web application is made for disabled users, the accessibility tools like High Contrast and Screen Reader must work perfectly. The V-Model gives equal importance to testing as it does to coding. By following this model, I was able to test every small module (like the user login, PDF generation, and survey

submission) step-by-step. This ensured that the final web application is reliable and has no major bugs.

2.6. WORK PLAN

Building a complete web application from zero is a big task. If I just started coding randomly without a schedule, I would have easily missed my deadlines or forgotten important features. To avoid this, I created a detailed work plan before starting the actual development.

I divided the entire project into smaller, easy-to-manage phases. Instead of looking at it as one massive software, I treated it as step-by-step goals. Having a proper work plan helped me track my weekly progress and kept me motivated because I could clearly see what I had completed and what was left to do.

Here is how I divided the main phases of my work plan:

2.6.1. Planning and Requirement Gathering

In the first few weeks, I spent my time understanding what features I needed to build. I researched web accessibility standards (WCAG) to understand how screen readers and voice typing actually work. I also finalized the core requirements for the Admin, Creator, and Respondent panels.

2.6.2. System and Database Design

Once the requirements were clear, I moved to the design phase. I created the database tables in MySQL, planned the relationships between users, surveys, questions, and answers. I also designed the basic wireframes and User Interface (UI) layout so I knew exactly how the web pages would look.

2.6.3. Development and Coding

This was the longest phase of my work plan. I started by writing the HTML and CSS for the frontend. Then, I wrote the PHP scripts to connect the frontend with the MySQL database. After the basic system was working, I integrated the JavaScript accessibility features (Voice to Text, High Contrast, Color Blind modes, etc.).

2.6.4. Testing and Bug Fixing

In the final phase, I thoroughly tested the application. I checked all the forms to make sure data is saving correctly, tested the branching logic to see if conditional questions appear properly, and most importantly, tested the accessibility tools on different browsers to ensure they work smoothly for disabled users.

2.7. PROJECT STRUCTURE

To successfully build and deliver the "Access Form" web application, it was very important to have a clear structure of who is doing what and when it will be finished. Managing a complete software system requires defining the team roles properly and following a strict schedule so that deadlines are not missed.

2.7.1. Team Structure

Since this web application was independently developed as a solo project, the entire team structure consisted of a single full-stack developer. Building a complete system from scratch by myself was challenging, but it gave me full control over how the frontend interacts with the backend. To keep things organized, I divided my responsibilities into four main software engineering roles:

- **Project Manager:** I was responsible for planning the requirements, deciding the features, and making sure the project followed the V-Model methodology.
- **Frontend Developer:** I designed the user interface using HTML, CSS, and JavaScript. My main focus here was integrating the accessibility features like the Web Speech API (for screen readers and voice typing) and creating the Color-Blind modes.
- **Backend Developer:** I wrote the core logic using Core PHP and managed the data using MySQL. I created the database connections using PDO to ensure the system is secure and can handle dynamic surveys and branching logic.
- **Quality Assurance (QA) Tester:** I acted as the tester to verify all the accessibility rules (WCAG 2.1 compliance) and to ensure that the PDF/CSV export features worked perfectly on different browsers.

2.7.2. Project Schedule (Submission Calendar)

To make sure the project was completed smoothly, I followed a strict schedule based on the milestones defined in my initial planning phase. The entire development lifecycle spanned over around six months. Here is the detailed schedule of how the project phases were completed:

- **Phase 1: Software Requirements Specification (Nov 2025)**
During the first month, I focused purely on gathering requirements. I finalized what the Admin, Creator, and Respondent panels would do and documented all the functional and non-functional requirements.
- **Phase 2: System Design Phase (Dec 2025 – Jan 2026)**
In these two months, I designed the architecture of the web application. I created the Database models (ERD), Use Cases, and Sequence Diagrams to map out how the data will flow between the user and the server.
- **Phase 3: Prototyping (Feb 2026)**
I developed a basic working prototype of the frontend. I designed the UI layouts and tested the drag-and-drop concepts for survey creation to see if the interface was user-friendly.

➤ **Phase 4: Final Deliverable - Coding & Testing (Mar 2026 – May 2026)**

This was the final and most intensive coding phase. I developed the PHP backend, connected the MySQL database, and wrote the JavaScript logic for branching questions and voice-to-text features. With the final deadline shifted to May 11, 2026, I used the extra days to perform rigorous system testing and fix accessibility bugs before the final deployment of the application.

GANTT CHART

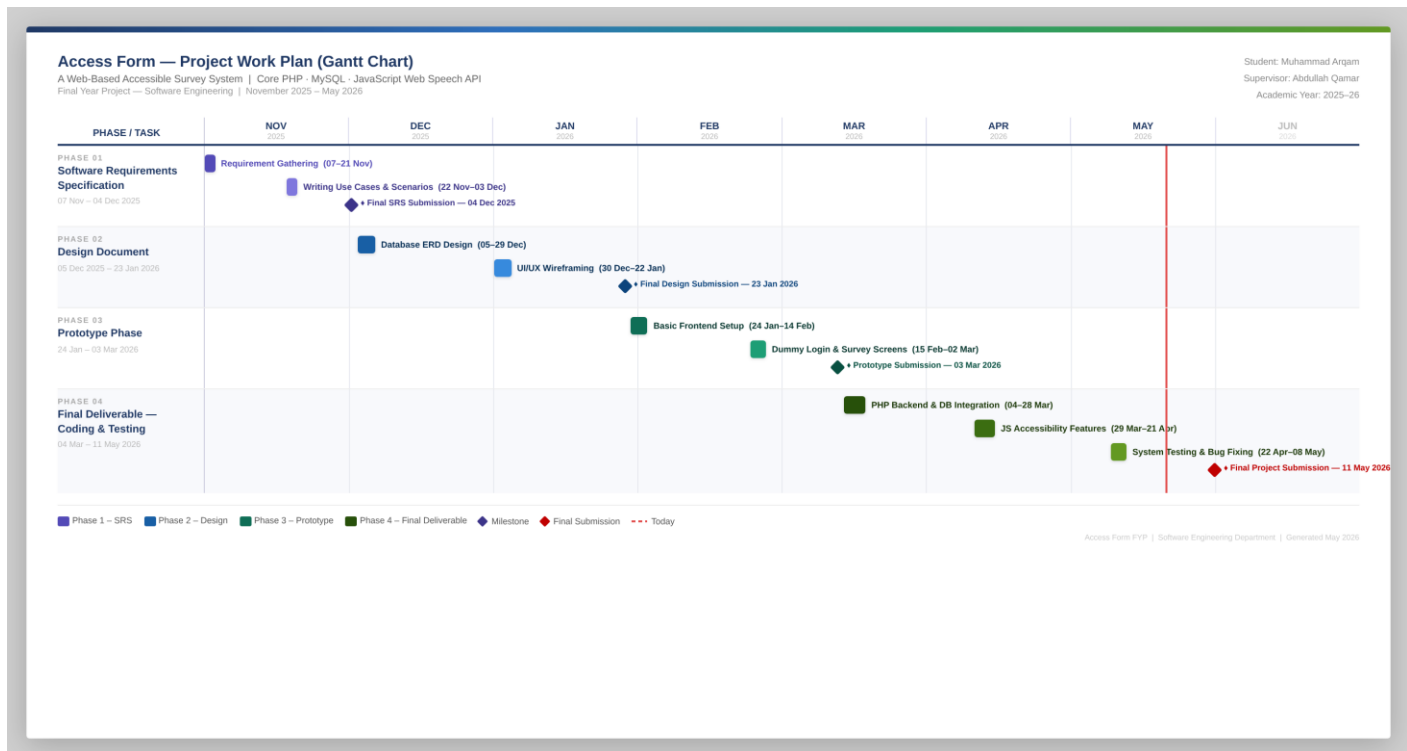


Figure 2.2 - ACCESS FORM - PROJECT PLAN (GANTT CHART)

CHAPTER 3

Designing the Project

3.1. INTRODUCTION

After completing the requirement gathering and planning phases, the next big step is designing the system. If a developer starts writing code without a proper design, it leads to a lot of confusion and bugs later on. This chapter covers the complete design phase of the "Access Form" web application.

Before working on the PHP backend or MySQL database, I needed a clear picture of how different parts of the system would talk to each other. In this chapter, I have included several design models to explain the system logic. I created Sequence Diagrams to show the step-by-step flow of actions, like what happens exactly when a user logs in, creates a survey, or submits their answers. I also created a Class Diagram to structure the overall logic and an Entity Relationship Diagram (ERD) to plan how the database tables will be connected.

Along with the logical design, I also worked on the visual part, which is the Graphical User Interface (GUI). Since the main goal of this web application is accessibility, I had to make sure the screens were designed in a very simple and clean way. This was important so that disabled users could navigate the forms easily using just a keyboard. Overall, this entire chapter acts as a blueprint that I followed strictly during the actual coding phase.

3.2. PURPOSE

The main purpose of this design chapter is to create a clear roadmap before starting the actual coding work. When we build a web application, we cannot just guess the database tables or user interfaces on the spot. We need a proper plan to follow. If a developer skips the design phase, they usually end up writing bad code and have to change the database structure again and again.

For my project, "Access Form", the design phase is even more important. Since I have to implement features like branching logic (where a question depends on the previous answer) and different accessibility tools, my database and system flow need to be perfect. The purpose of drawing sequence diagrams and the Entity Relationship Diagram (ERD) is to understand how the Admin, Form Creator, and Respondent will interact with the system step-by-step. It helps me see how the data will move from the frontend forms to the MySQL database.

Another big purpose of this chapter is to finalize the User Interface (UI). I needed to design the screens first so I could make sure the buttons are easy to click and the colors are safe for color-blind people. In short, the purpose of this whole chapter is to solve all the logical and visual problems on paper first, which will save a lot of my time during the actual PHP and JavaScript coding phase.

3.3. SCOPE

The scope of this design chapter is strictly limited to planning the architecture, database, and visual interfaces of the "Access Form" web application. It does not include any actual PHP, SQL, or JavaScript coding. Instead, it covers all the structural blueprints that are required before the development phase begins.

First, the scope includes the dynamic model of the system. For this, I have created multiple Sequence Diagrams. These diagrams will show the step-by-step flow of specific tasks, such as how a user logs in, how a form creator adds branching questions, and how the system saves a respondent's answers.

Second, this chapter covers the data storage plan. I have included a Class Diagram to define the system's structure and an Entity Relationship Diagram (ERD) for the database. The ERD is a very important part of this scope because it clearly defines the exact tables (like users, surveys, questions, and responses) and their connections that I will create in the MySQL database.

Finally, the scope covers the Graphical User Interface (GUI) design. I have included the screen designs for the Admin, Form Creator, and Respondent panels. The focus of these interface designs is purely on web accessibility, ensuring that the layout is clean, colors are safe, and the screens are easy to navigate for disabled users.

3.4. DEFINITIONS, ACRONYMS AND ABBREVIATIONS

In this design chapter, I have used several technical terms related to software modeling and database planning. To make it easy to understand for anyone reading this document, I have explained these key terms below, along with how they are used in my project:

3.4.1. UML (Unified Modeling Language)

- **Definition:** A standard drawing language used by developers to create visual Models of a software system.
- **Relevance in Project:** I used UML rules to draw the Sequence Diagrams and Class Diagram so that the system logic is universally easy to understand.

3.4.2. ERD (Entity Relationship Diagram)

- **Definition:** A visual chart that shows how different tables in a database are related and connected to each other.
- **Relevance in Project:** This is the blueprint for my MySQL database. It clearly shows how the "User" table is linked to the "Surveys" they create, and how "Questions" are connected to "Answers".

3.4.3. Sequence Diagram

- **Definition:** A step-by-step diagram that shows how different parts of a system talk to each other over time to complete a specific task.
- **Relevance in Project:** I created these diagrams to plan the exact background flow for important actions, like what happens step-by-step when a user clicks the login button or submits a survey.

3.4.4. Class Diagram

- **Definition:** A static diagram that shows the structure of the system by dividing it into classes, their properties, and relationships.
- **Relevance in Project:** It gives a high-level view of all the main system components like Admin, Creator, Respondent, and Accessibility Settings.

3.4.5. PK and FK (Primary Key & Foreign Key)

- **Definition:** Database terms. A Primary Key is a unique ID for a record in a table, and a Foreign Key is used to link that record to another table.
- **Relevance in Project:** Used heavily in my database design. For example, a Creator's ID (PK) is saved in the Survey table as a Foreign Key (FK) to know who created that specific survey.

3.4.6. GUI (Graphical User Interface)

- **Definition:** The visual screens, buttons, menus, and forms that the user actually sees and clicks on the screen.
- **Relevance in Project:** Designing the GUI was a big focus in this chapter because the interface of "Access Form" needs to be very simple, clean, and accessible for disabled users.

3.4.7. Multi-Tier Architecture

- **Definition:** A software design method where the presentation (UI), application logic, and data storage are managed in separate layers.
- **Relevance in Project:** I designed the system in three tiers (Frontend, Backend, Database) so that the PHP code does not get messy and the accessibility features can work smoothly on the frontend.

3.5. DYNAMIC MODEL: SEQUENCE DIAGRAMS

In software engineering, a dynamic model helps us understand the behavior of the system over time. While static models just show the structure (like database tables), dynamic models show what actually happens in the background when a user clicks a button or performs an action. To represent this dynamic behavior for the "Access Form" project, I have used Sequence Diagrams.

A Sequence Diagram basically shows the step-by-step flow of messages between different parts of the system. For example, when a Form Creator tries to add a new question, the sequence diagram shows how the request travels from the User Interface (HTML/CSS), gets processed by the backend logic (PHP), and finally gets saved in the database (MySQL). If an error occurs, it shows how the error message travels back to the user.

I have created sequence diagrams for all the major functionalities of my web application to make the coding phase easier. By drawing these diagrams, I knew exactly how my frontend and backend would talk to each other. Below are the sequence diagrams for the core features of the "Access Form" system:

3.5.1. User Authentication (System Wide)

This sequence diagram shows how a user logs into the system. The PHP backend verifies the hashed password and also fetches the user's saved accessibility settings from the database before redirecting them to their dashboard.

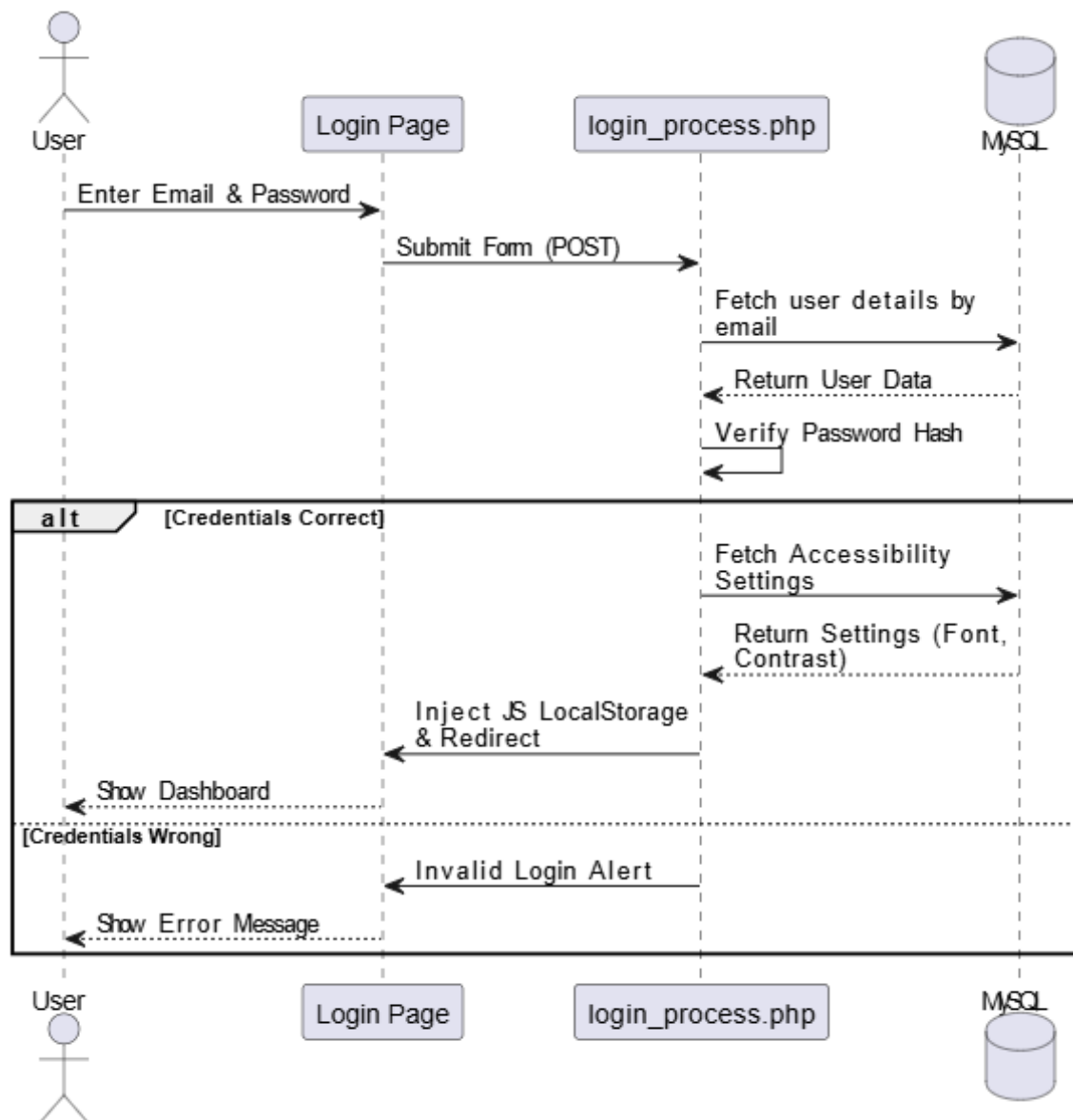


Figure 3.1 - User Authentication

3.5.2. Manage Users (Admin Panel)

This diagram shows the Admin's action to enable, disable, or delete a user. The request is handled by the “user_action.php” file in the code.

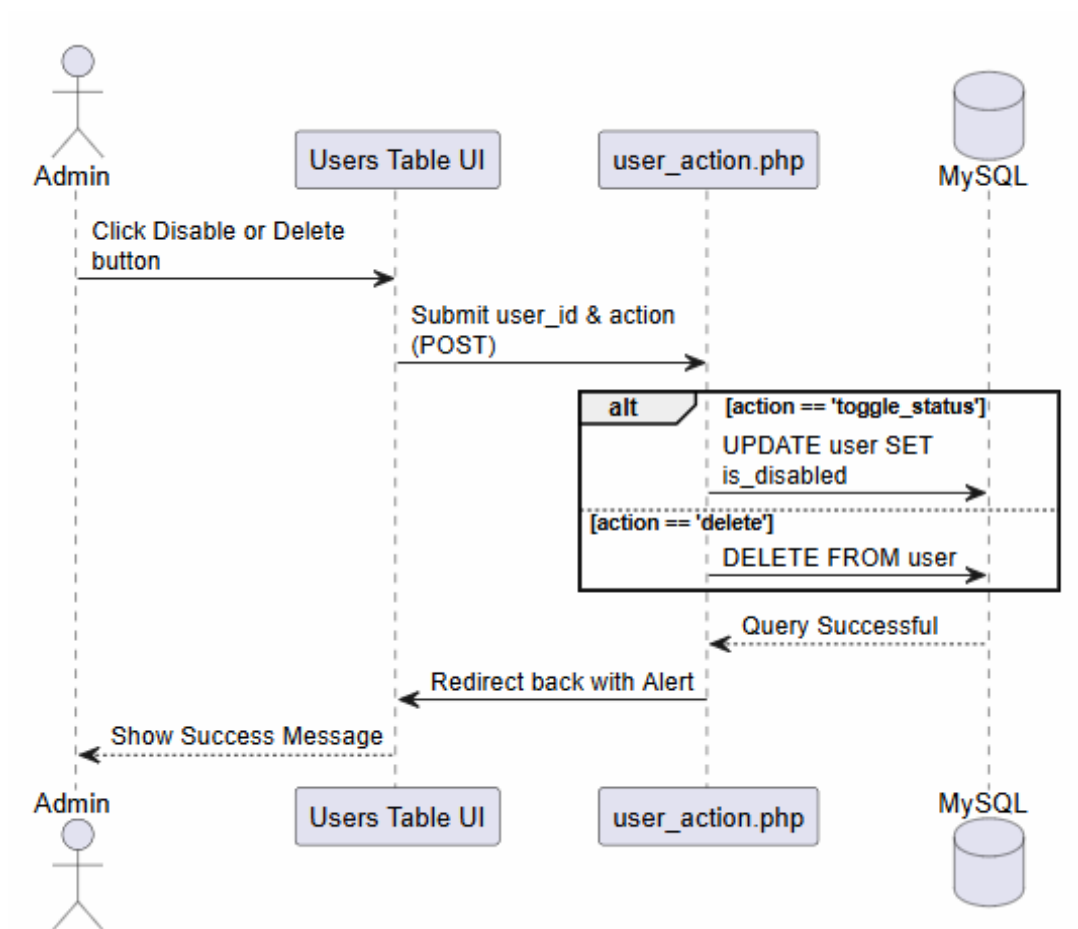


Figure 3.2 - Manage Users

3.5.3. Create New Survey (Creator Panel)

Here, the Form Creator starts a new survey. The system saves it as a 'Draft' first and returns a new Survey ID to open the form builder.

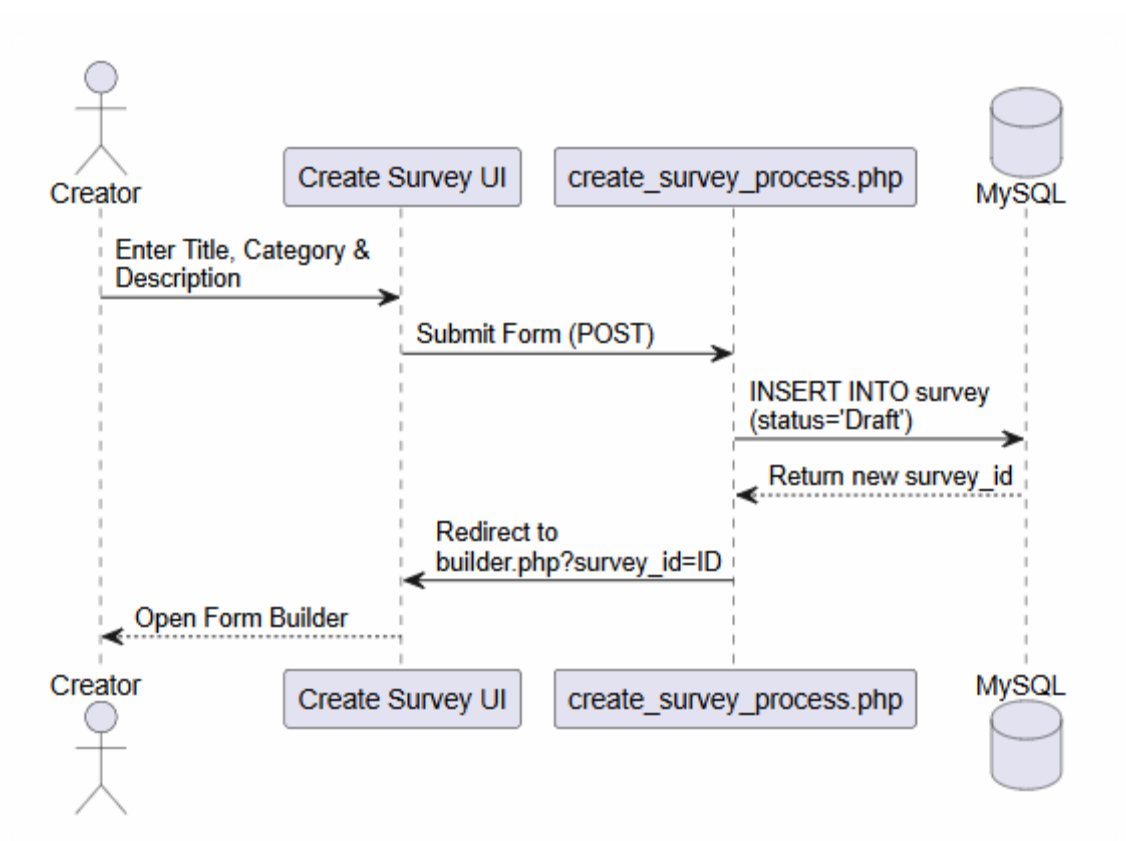


Figure 3.3 - Create New Survey

3.5.4. Build Survey / Add Question (Creator Panel)

This diagram explains how the Creator adds questions and branching logic into the drafted survey.

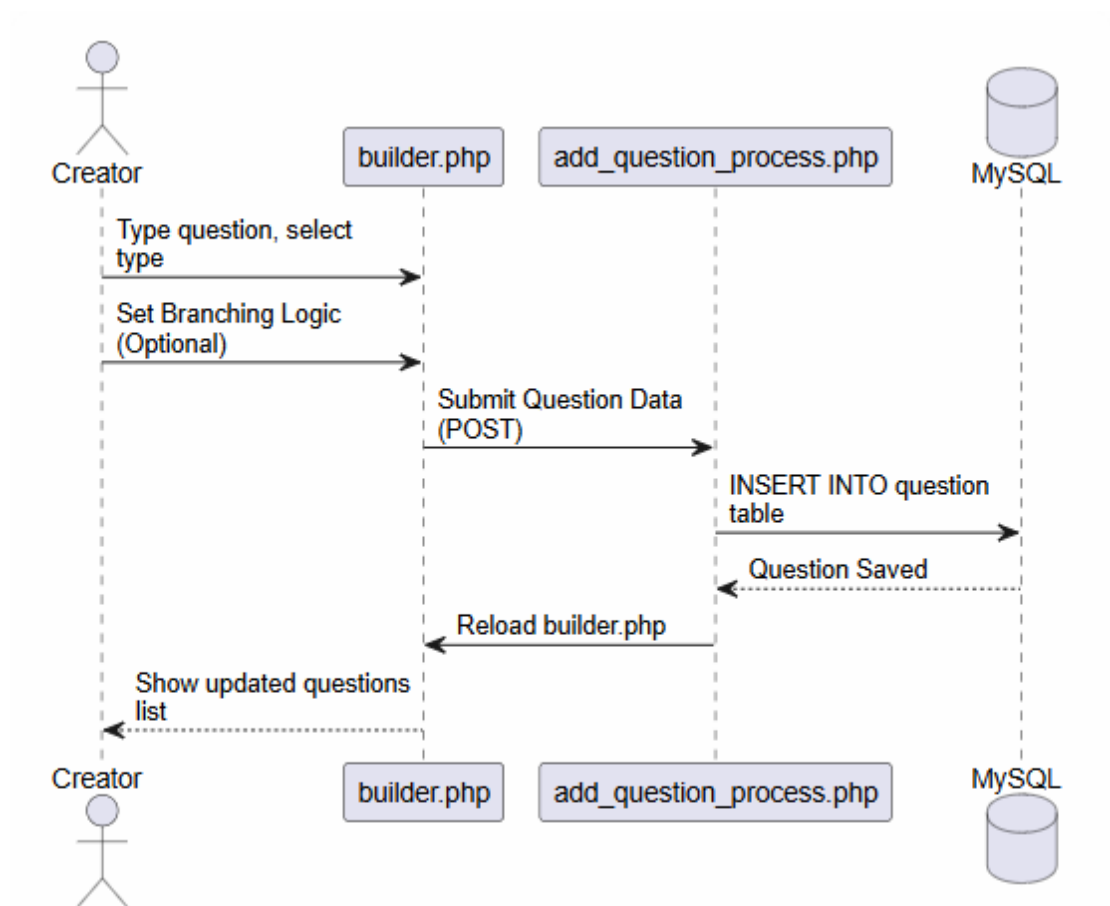


Figure 3.4 - Build Survey / Add Question

3.5.5. Publish Survey & Generate Link

This shows how the Creator finalizes the survey. The backend checks if questions exist before publishing, and then JavaScript copies the public link to the clipboard.

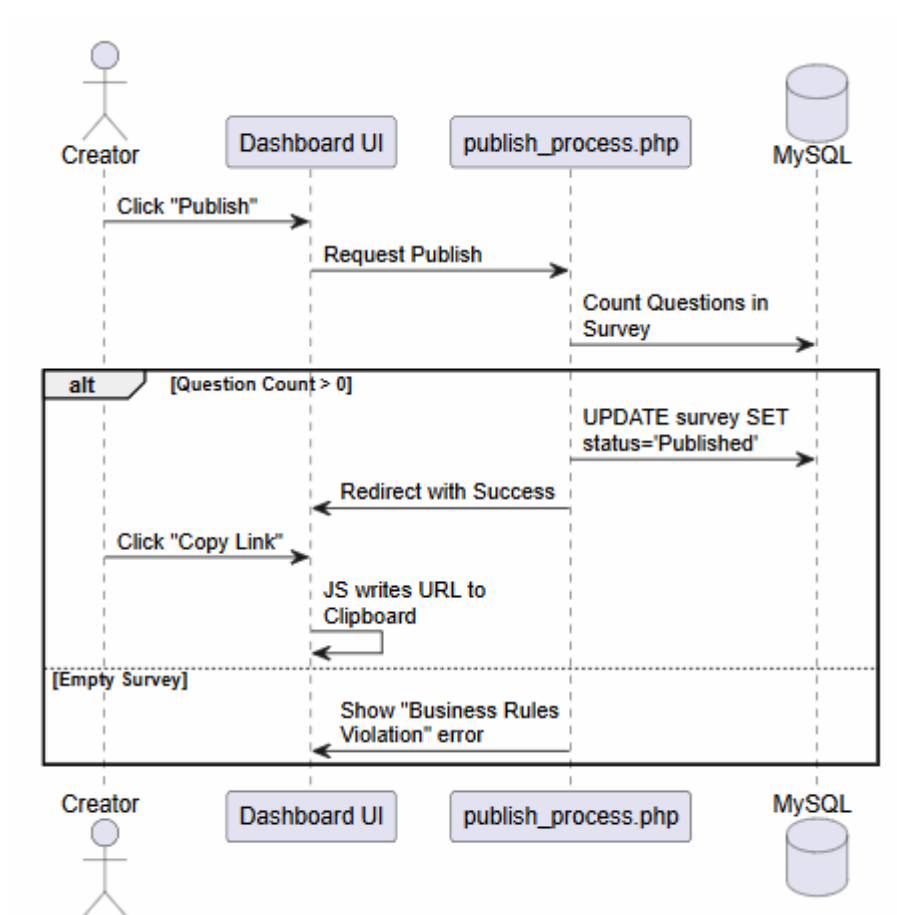


Figure 3.5 - Publish Survey & Generate Link

3.5.6. Fill & Submit Survey (Respondent Panel)

This diagram maps out the respondent's journey, including the use of the Web Speech API (Voice typing) to fill the form.

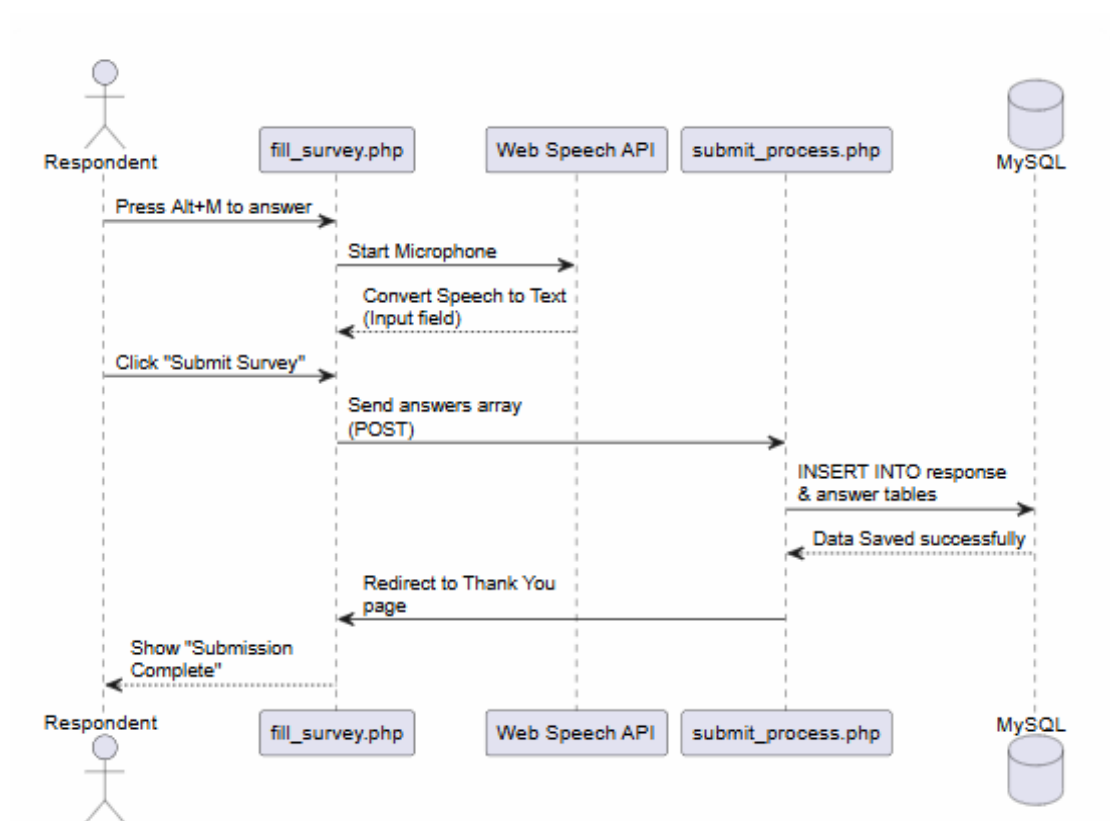


Figure 3.6 - Fill & Submit Survey

3.5.7. View & Export Responses

This shows how Admin or Creator views the results. It includes the html2pdf library logic for PDF and PHP headers for Excel generation.

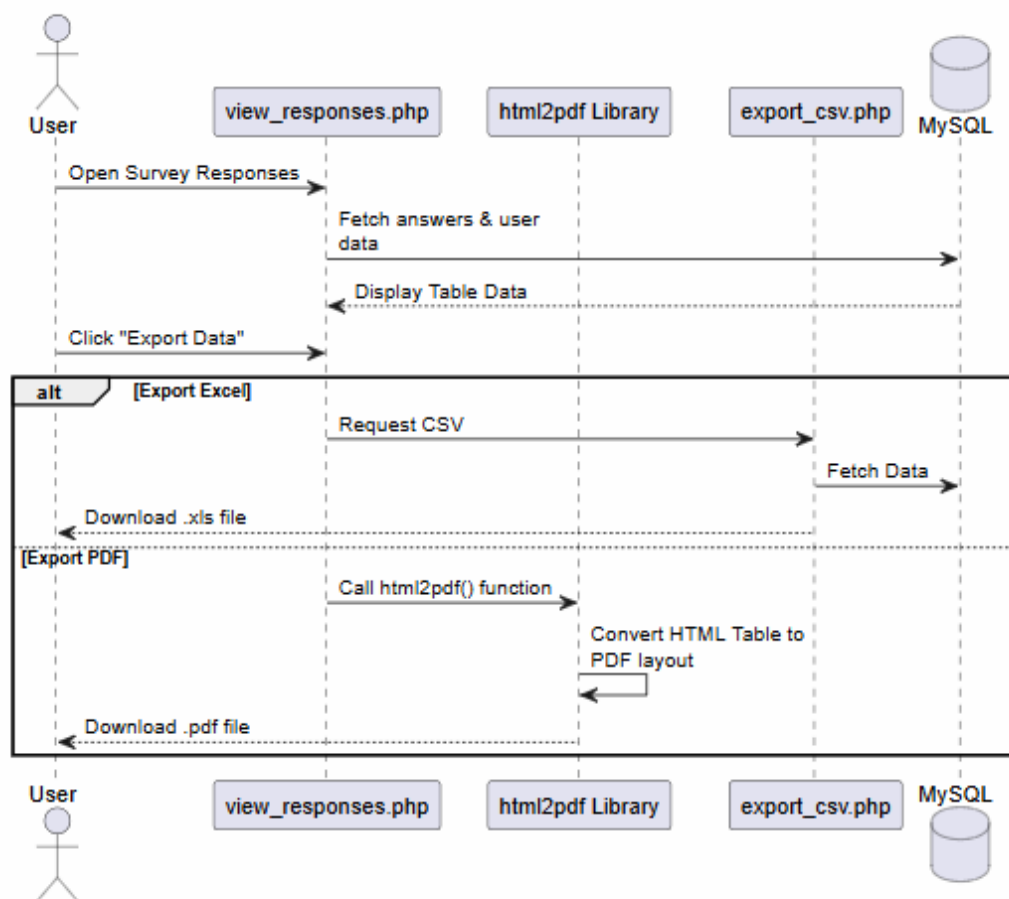


Figure 3.7 - View & Export Responses

3.5.8. Configure Accessibility Settings

This represents how the accessibility menu works across the system. It uses “accessibility.js” to change the UI instantly and AJAX to save it in the database.

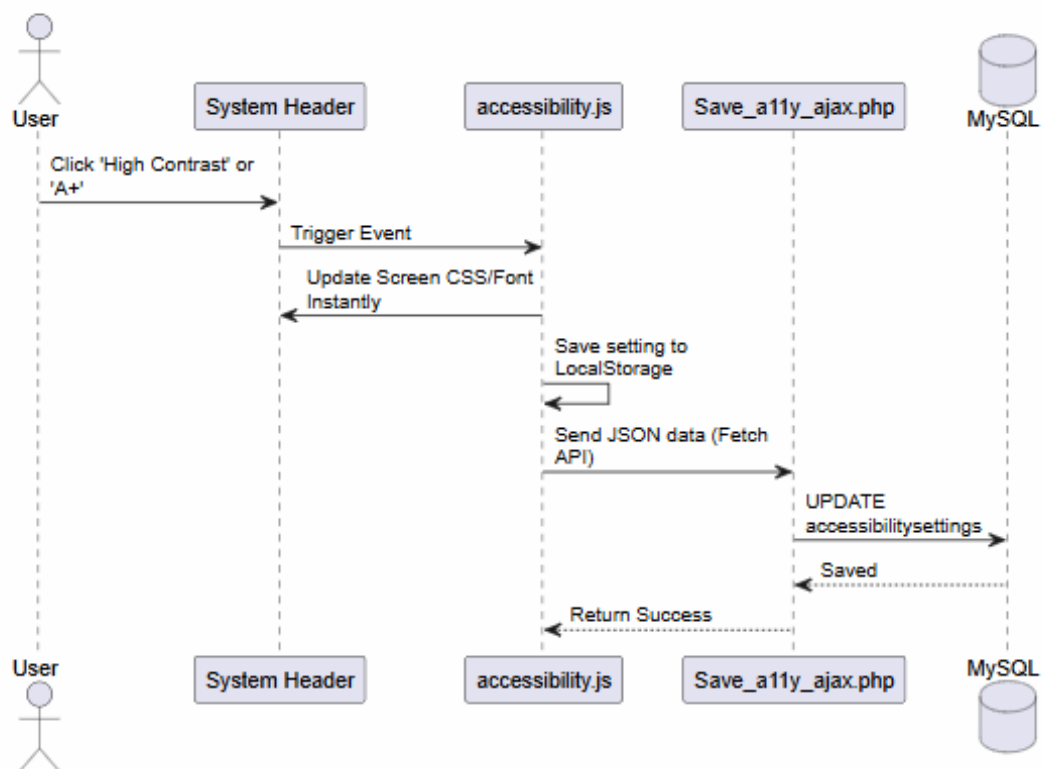


Figure 3.8 - Configure Accessibility Settings

3.6. OBJECT MODEL/LOGICAL MODEL: CLASS DIAGRAM

A Class Diagram is a static structure that shows the main building blocks of the system. Even though I developed this web application using Core PHP, it is still very important to logically map out the main entities of the system to understand how they interact with each other.

In this diagram, I have created classes based on the core database tables of the "Access Form" project, such as User, Survey, Question, Response, Answer, and AccessibilitySettings. Inside each class, I have listed the attributes (the data it stores, like email, password, or question text) and the operations (the actions performed on it, like login, createSurvey, or saveAnswer). This logical model gives a very clear panoramic view of the entire system architecture.

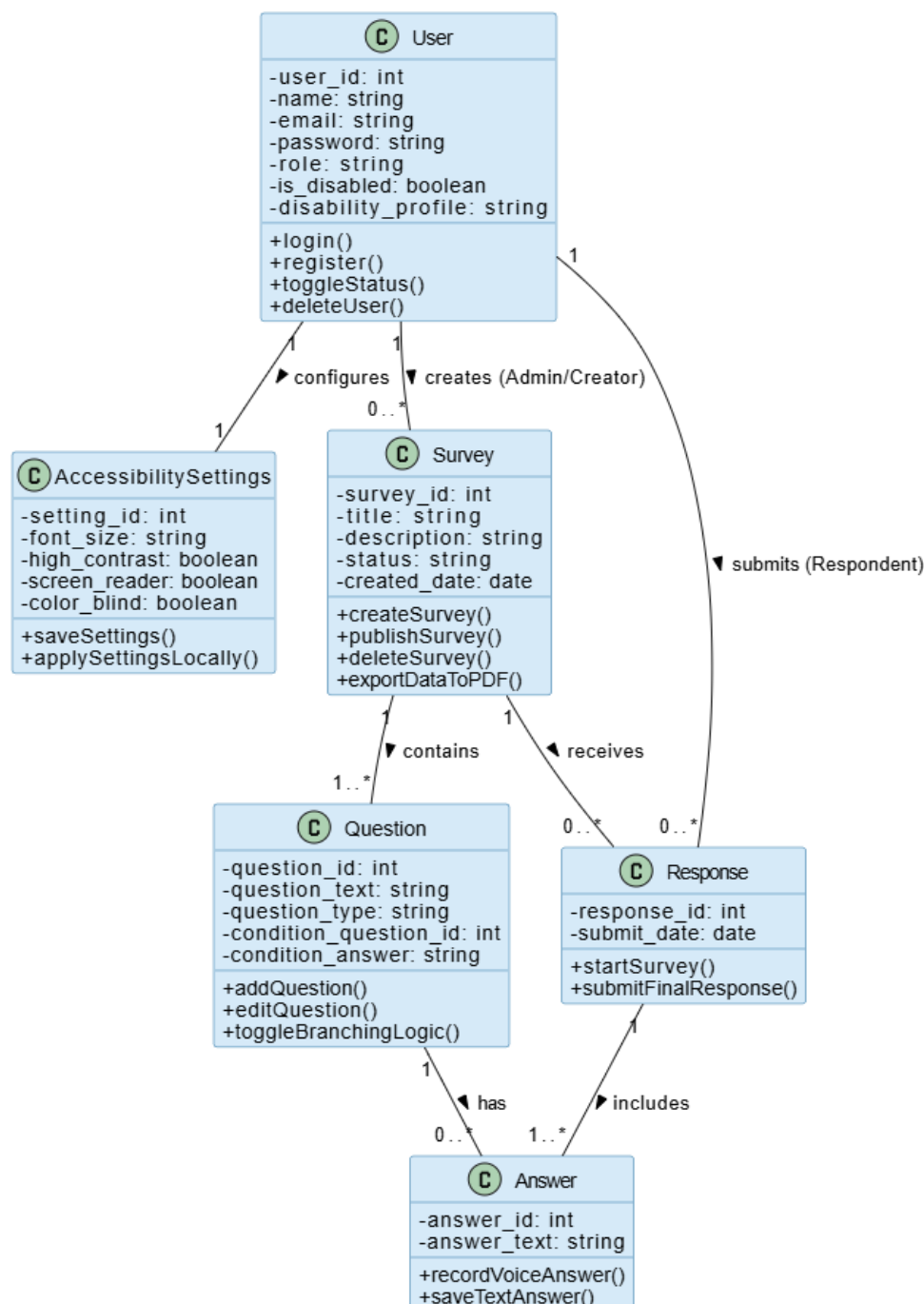


Figure 3.9 - Class Diagram

3.7. DATABASE MODEL (DATABASE DIAGRAM)

Every web application needs a solid database to store and manage data. If the database design is messy, the whole PHP backend will suffer. To plan the database for the "Access Form" system, I created an Entity Relationship Diagram (ERD).

An ERD shows all the tables in the database, what columns they have, and how they are connected to each other using Primary Keys (PK) and Foreign Keys (FK).

In my project, the main entity is the **User** table, which stores details of the Admin, Creators, and Respondents. It is directly connected to the **AccessibilitySettings** table so that every user's specific disability profile (like high contrast or font size) is saved.

The **Survey** table holds the survey details and connects to the **Question** table. When a respondent fills a survey, their answers are stored in the **Response** and **Answer** tables. By creating this ERD first, I was able to write the exact MySQL queries easily without any confusion during the development phase.

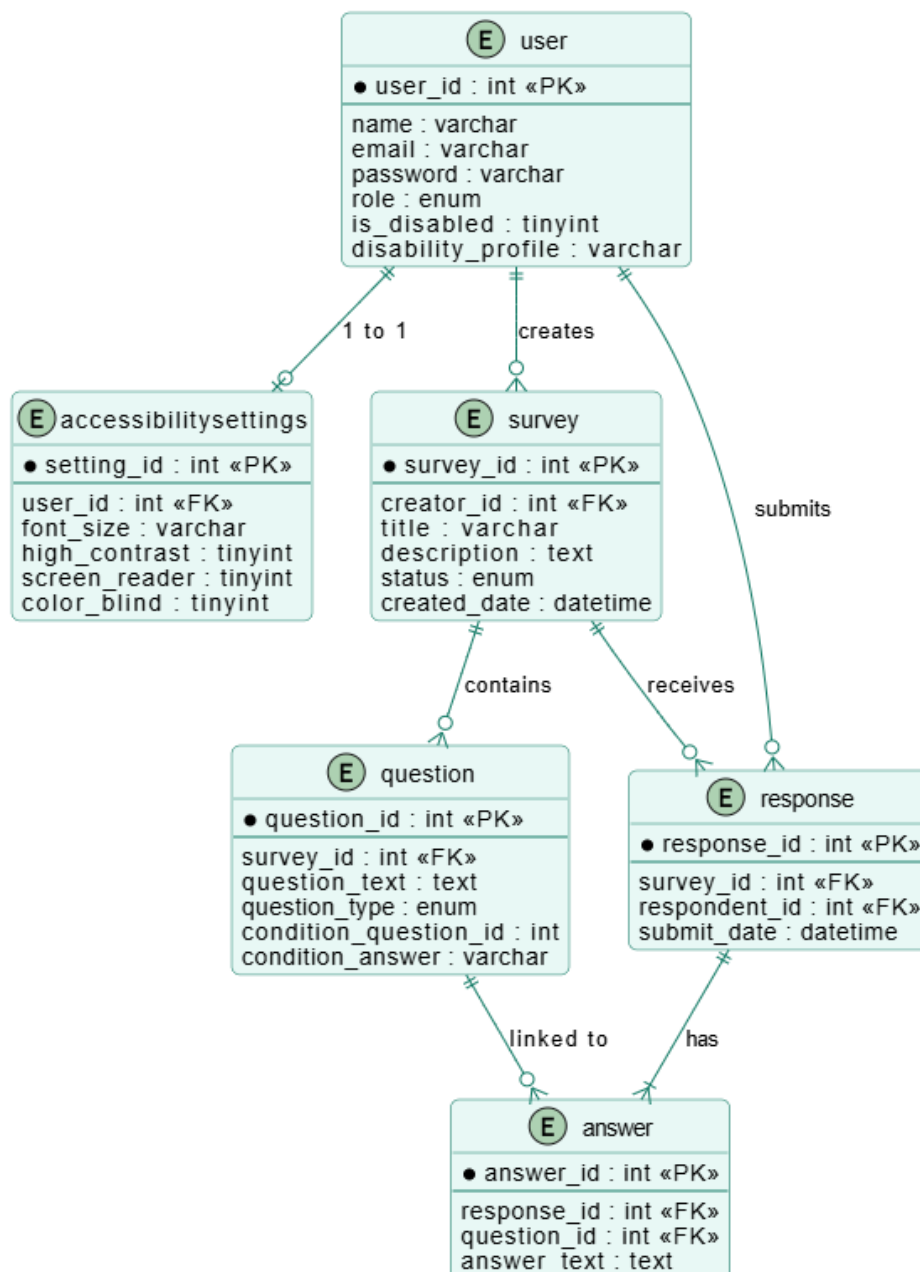


Figure 3.10 - Entity Relationship Diagram

3.8. GRAPHICAL USER INTERFACES

The Graphical User Interface (GUI) is basically the screens that users see and interact with. For the "Access Form" project, the interface design was the most important part because this web application is specifically made for disabled and elderly people.

If the screens are complicated or have very small text, visually impaired users will not be able to use the system. Because of this, I kept the design very clean and simple. I used big buttons, clear fonts, and simple colors. I also added a top menu on every screen so users can quickly change the text size or switch to a dark mode (High Contrast) if their eyes are hurting. Another important thing I designed was the keyboard focus. Whenever a user presses the "Tab" key, a thick border appears around the buttons so they exactly know where they are on the screen.

Below are some of the main interface designs of my web application. These include the screens for the Admin, the Form Creator, and the Respondent:

3.8.1. Authentication Screens:

➤ Registration Screen

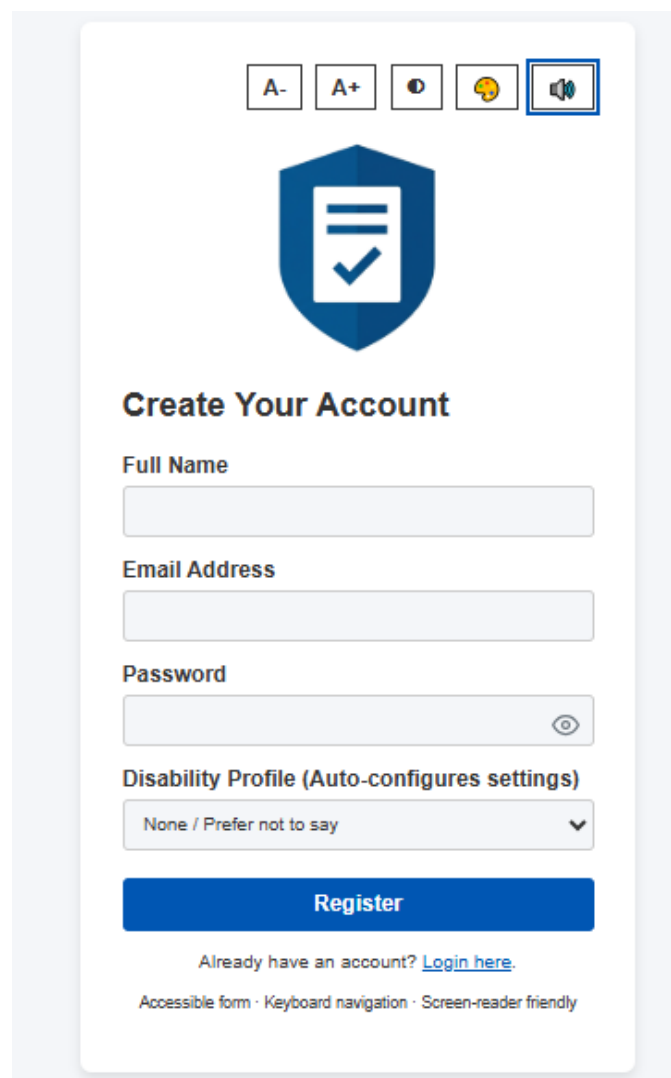
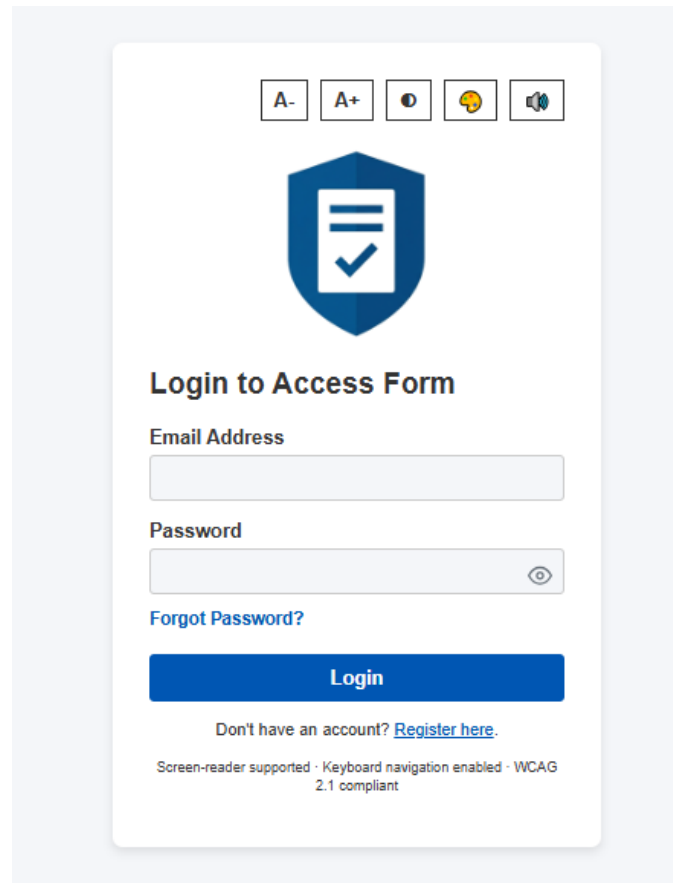
A screenshot of a web application's registration screen. At the top, there is a row of five icons: 'A-' for decreasing text size, 'A+' for increasing text size, a dark mode icon (a circle with a dot), a high contrast icon (a circle with a checkmark), and a speaker icon for audio feedback. Below these icons is a large blue shield icon containing a white document with a checkmark. The main heading is 'Create Your Account'. Below this are four input fields: 'Full Name', 'Email Address', 'Password' (with an eye icon for toggling visibility), and 'Disability Profile (Auto-configures settings)' (a dropdown menu currently showing 'None / Prefer not to say'). A large blue 'Register' button is positioned below the input fields. At the bottom, there is a link 'Already have an account? Login here.' and a footer note 'Accessible form · Keyboard navigation · Screen-reader friendly'.

Figure 3.11 - Registration Screen

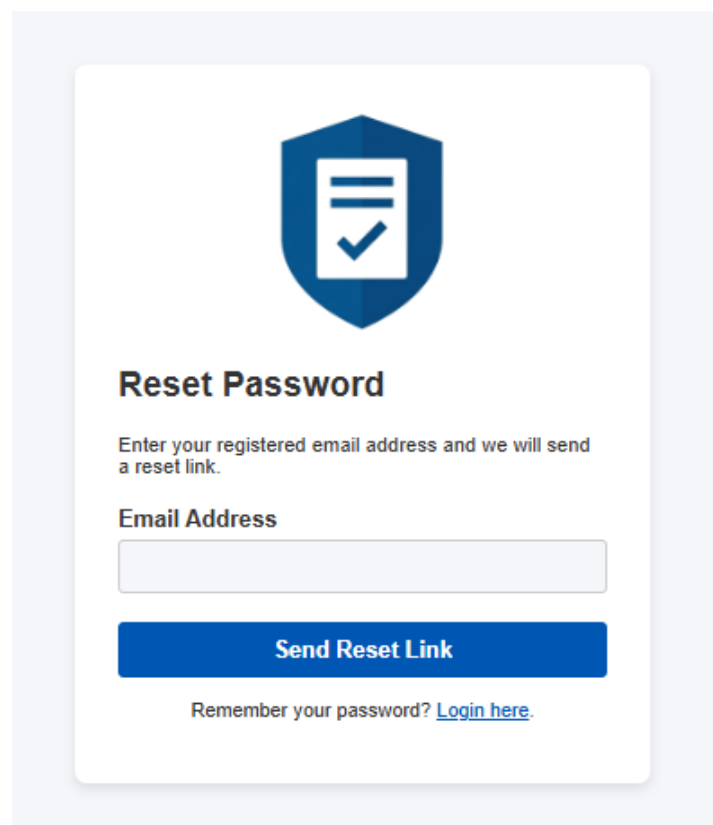
➤ Login Screen



The login screen features a white card on a light blue background. At the top of the card, there are five icons: 'A-' for font size reduction, 'A+' for font size increase, a dark circle for theme toggling, a speech bubble for help, and a speaker for audio. Below these icons is a blue shield icon containing a white document with a checkmark. The title 'Login to Access Form' is centered below the icon. The form includes an 'Email Address' label and a text input field, followed by a 'Password' label and a password input field with an eye icon for toggling visibility. A 'Forgot Password?' link is positioned below the password field. A prominent blue 'Login' button is centered below the form fields. At the bottom of the card, there is a link 'Don't have an account? Register here.' and a footer line stating 'Screen-reader supported · Keyboard navigation enabled · WCAG 2.1 compliant'.

Figure 3.12 - Login Screen

➤ Forgot / Reset Password Screen



The reset password screen features a white card on a light blue background. At the top of the card is a blue shield icon containing a white document with a checkmark. The title 'Reset Password' is centered below the icon. Below the title, a message reads 'Enter your registered email address and we will send a reset link.' This is followed by an 'Email Address' label and a text input field. A prominent blue 'Send Reset Link' button is centered below the input field. At the bottom of the card, there is a link 'Remember your password? Login here.'

Figure 3.13 - Forgot/reset Password Screen

3.8.2. Admin Panel:

➤ Admin Dashboard

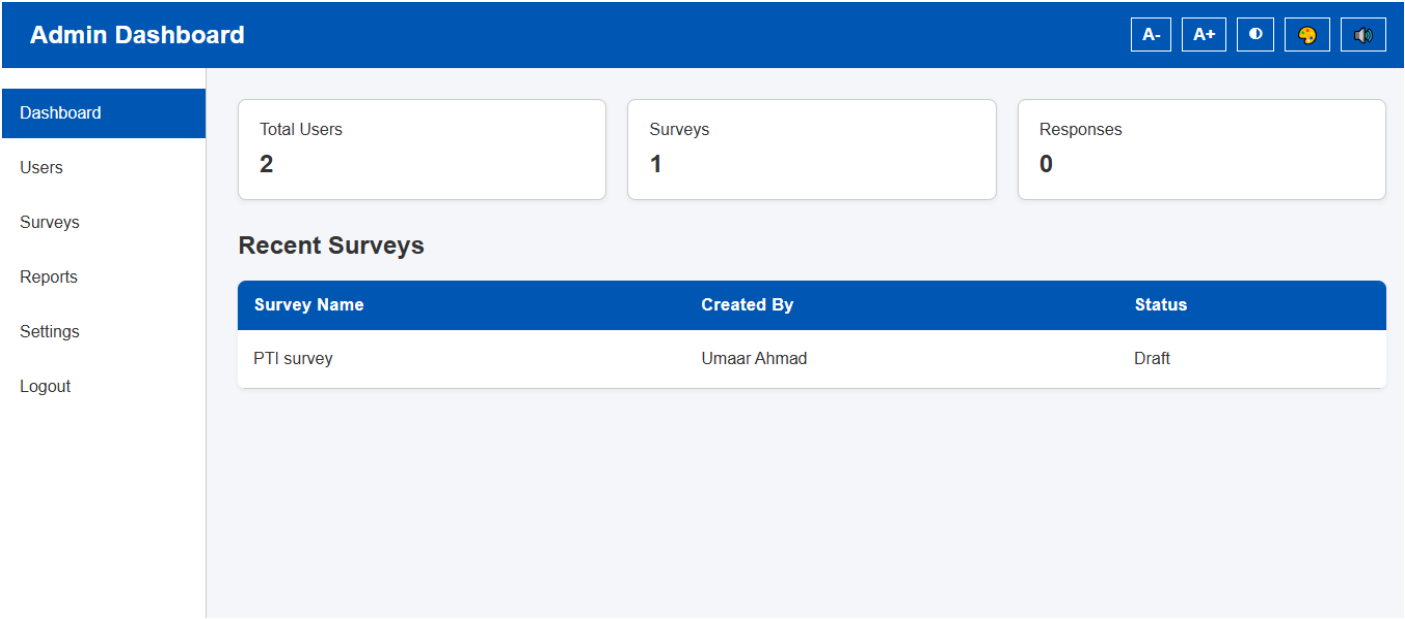


Figure 3.14 - Admin Dashboard Screen

➤ Manage Users Screen

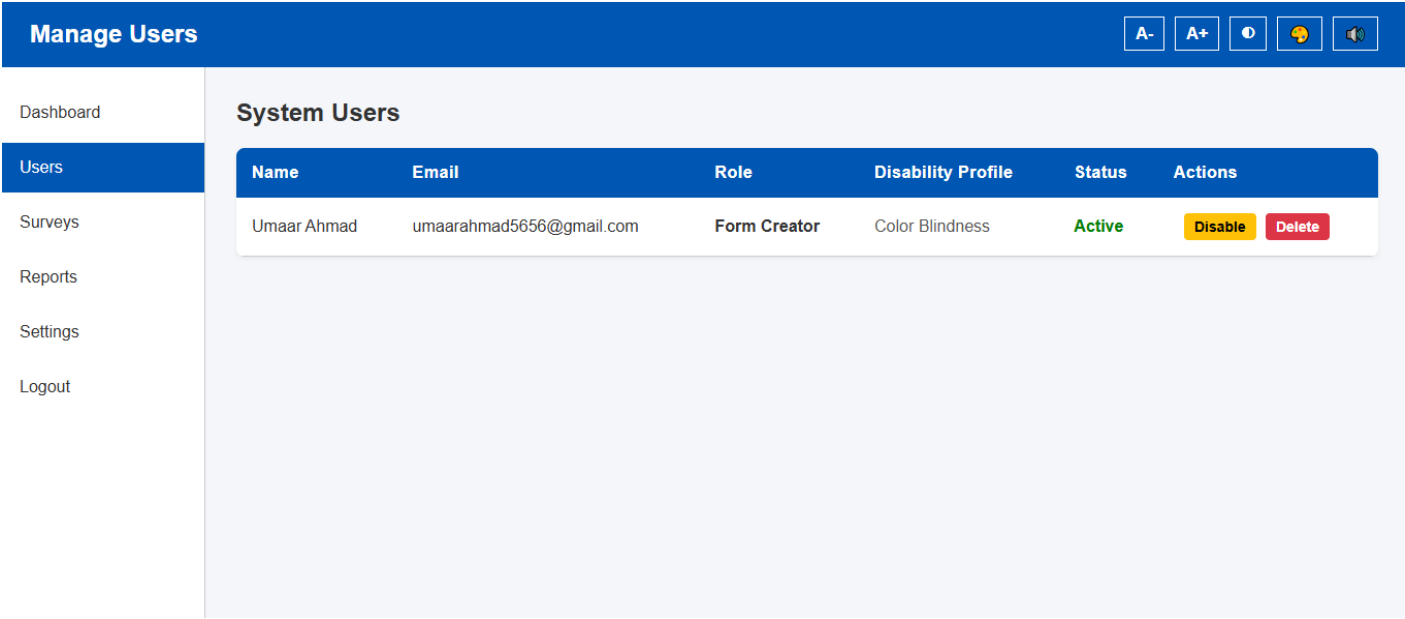


Figure 3.15 - Manage Users Screen

➤ System Reports / Export Screen

System Reports

Dashboard
Users
Surveys
Reports
Settings
Logout

Generate Survey Reports

Survey Name	Creator	Total Responses	Action
Teacher Evaluation	creator	1	View Data & Export

Figure 3.16 - System Reports / Export Screen

3.8.3. Form Creator Panel:

➤ Creator Dashboard

Form Creator Dashboard

Dashboard
Create Survey
My Surveys
Responses
Settings
Logout

My Total Surveys
1

My Recent Surveys

Survey Name	Status	Created Date	Actions
Teacher Evaluation	Published	04 May 2026	View Copy Link Delete

Figure 3.17 - Creator Dashboard Screen

➤ Create Survey Screen

Create Survey

Dashboard

Create Survey

My Surveys

Responses

Settings

Logout

Survey Title

Survey Description

Survey Category

Feedback

Create Survey

Fully accessible form · Keyboard friendly · Screen-reader supported · WCAG 2.1 compliant

Figure 3.18 - Create Survey Screen

➤ Form Builder Screen

Form Builder: PTI Survey

Dashboard

Form Builder

Preview

Publish

Logout

Add New Item

Text (Question or Section Title)

Type

Text Answer

Alt-Text (for screen readers)

Describe purpose or leave blank

Branching Logic (Optional)

Show this question ONLY IF a previous question has a specific answer.

Show this ONLY IF Question:

Always Show (No Condition)

Equals Answer (e.g., Yes, Agree, 5):

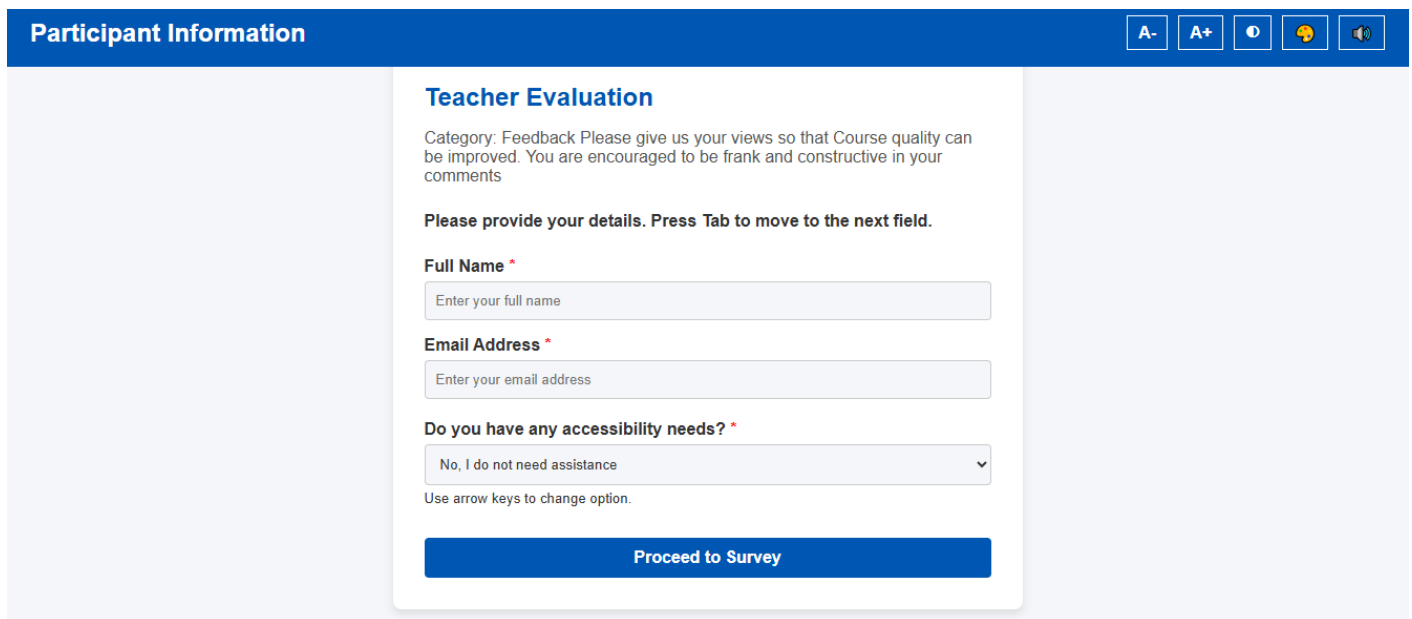
Type exact answer (Leave blank if no condition)

Add Item **Preview Form**

Figure 3.19 -Form Builder Screen

3.8.4. Respondent Panel:

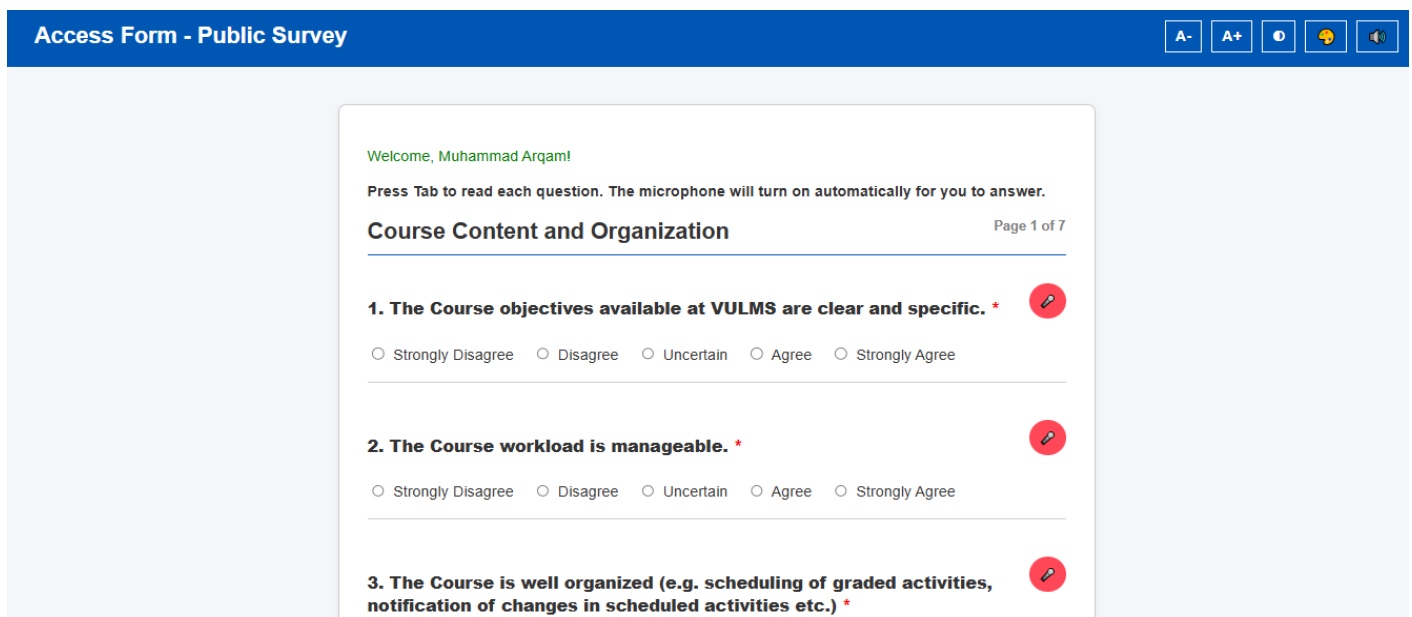
➤ Start Survey Screen



The screenshot shows the 'Start Survey Screen' with a blue header bar labeled 'Participant Information'. On the right side of the header are five icons: 'A-', 'A+', a circle with a dot, a speech bubble, and a speaker. The main content area is a white box titled 'Teacher Evaluation'. Below the title is a category description: 'Category: Feedback Please give us your views so that Course quality can be improved. You are encouraged to be frank and constructive in your comments'. A prompt says 'Please provide your details. Press Tab to move to the next field.' There are three input fields: 'Full Name *' with a placeholder 'Enter your full name', 'Email Address *' with a placeholder 'Enter your email address', and a dropdown menu for 'Do you have any accessibility needs? *' with the selected option 'No, I do not need assistance'. Below the dropdown is a hint: 'Use arrow keys to change option.' At the bottom of the white box is a blue button labeled 'Proceed to Survey'.

Figure 3.20 - Start Survey Screen

➤ Fill Survey Screen



The screenshot shows the 'Fill Survey Screen' with a blue header bar labeled 'Access Form - Public Survey'. On the right side of the header are five icons: 'A-', 'A+', a circle with a dot, a speech bubble, and a speaker. The main content area is a white box. At the top, it says 'Welcome, Muhammad Arqam!'. Below that is a prompt: 'Press Tab to read each question. The microphone will turn on automatically for you to answer.' The title 'Course Content and Organization' is followed by 'Page 1 of 7'. There are three questions, each with a red pencil icon to its right. Question 1: '1. The Course objectives available at VULMS are clear and specific. *' with radio buttons for 'Strongly Disagree', 'Disagree', 'Uncertain', 'Agree', and 'Strongly Agree'. Question 2: '2. The Course workload is manageable. *' with the same radio button options. Question 3: '3. The Course is well organized (e.g. scheduling of graded activities, notification of changes in scheduled activities etc.) *' with the same radio button options.

Figure 3.21 - Fill Survey Screen

➤ **Submission Complete / Thank You Screen**

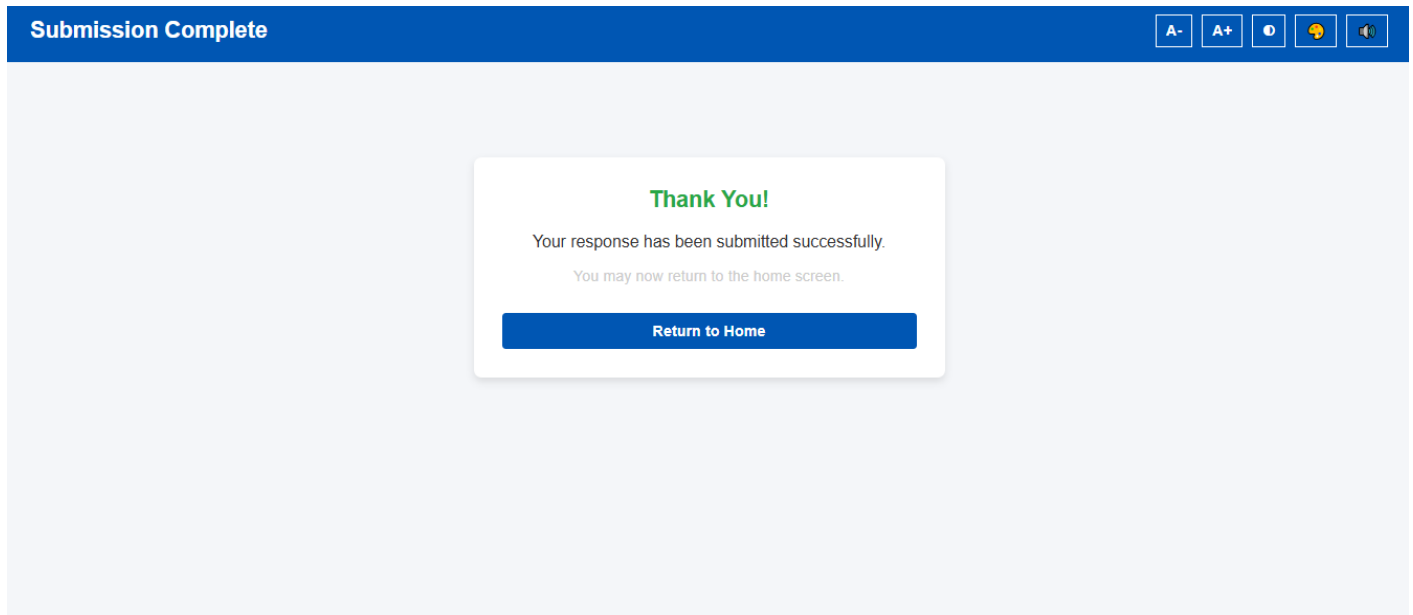


Figure 3.22 - Submission Complete / Thank You Screen

3.8.5. Accessibility Features

➤ **Default Mode Screen**

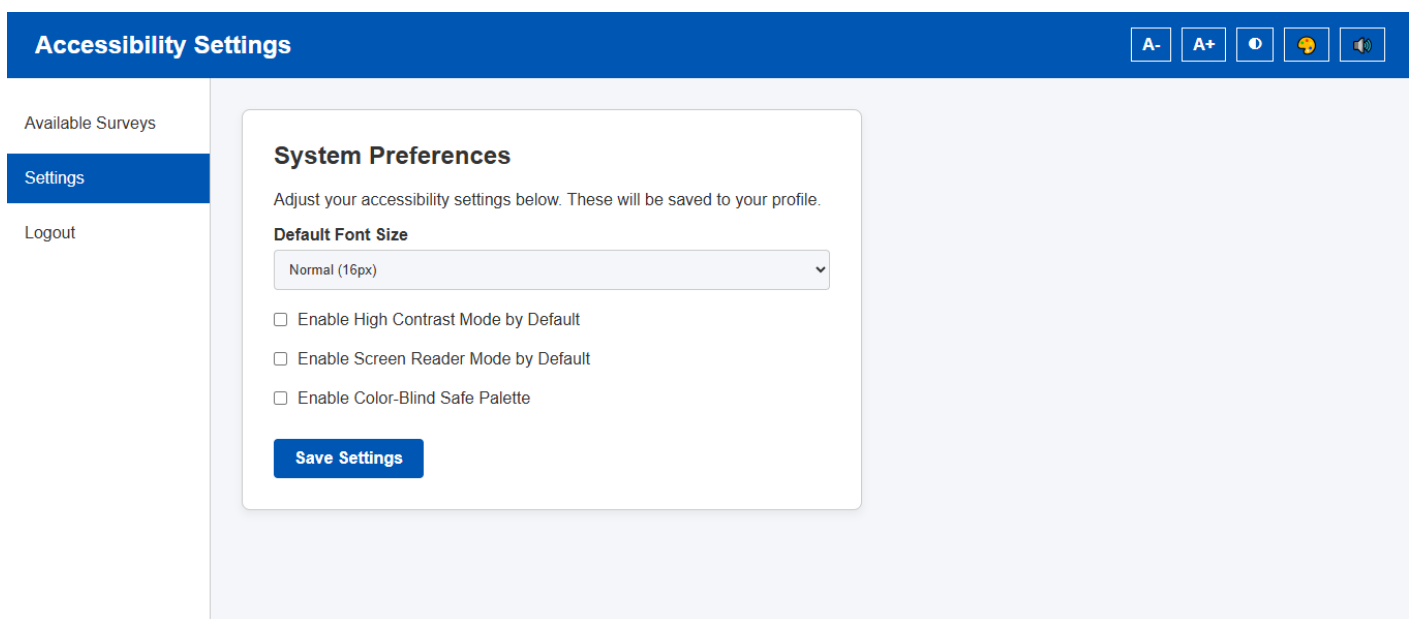


Figure 3.23 - Default Mode Screen

➤ **Dark Mode Screen (High Contrast)**

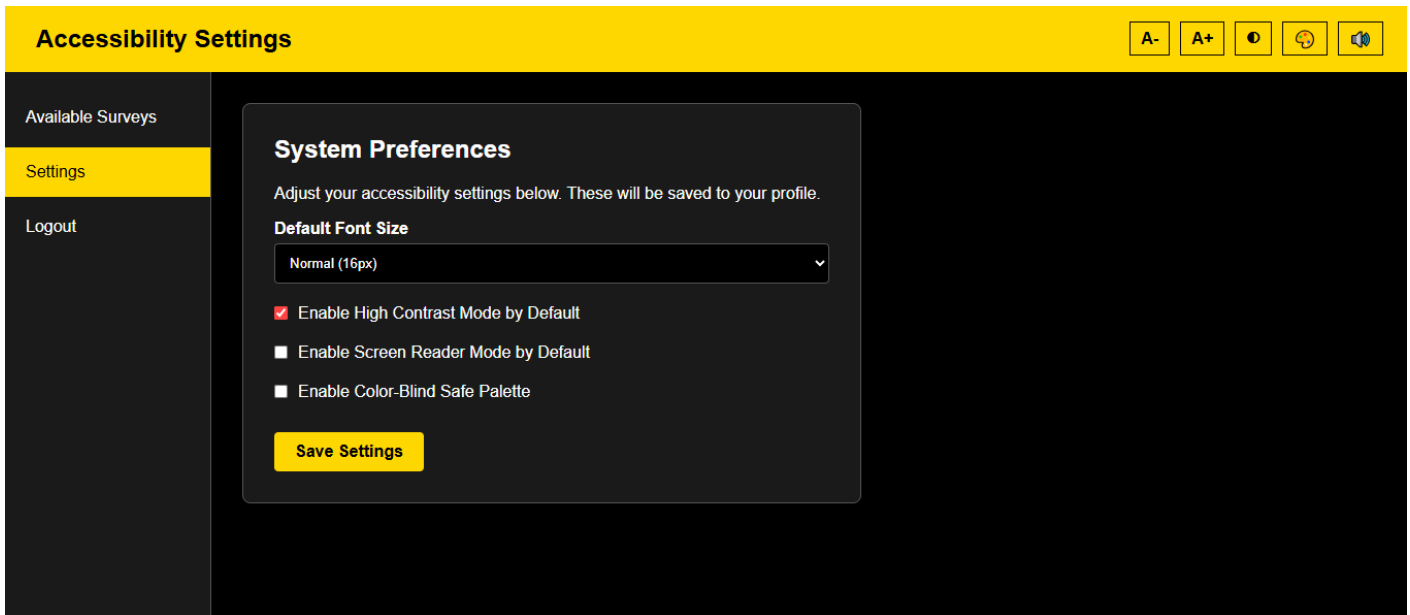


Figure 3.24 - Dark Mode / High Contrast Screen

➤ **Color-Blind Mode**

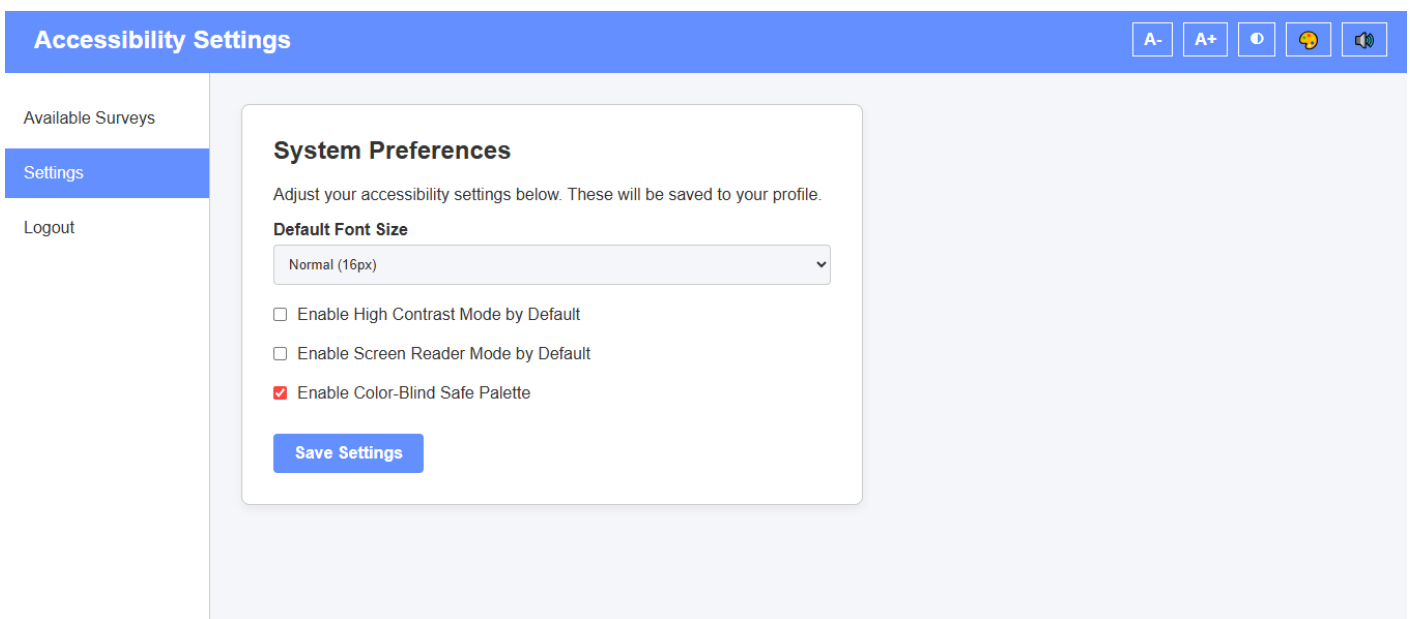


Figure 3.25 - Color-Blind Mode Screen

CHAPTER 4

Development

4.1. DEVELOPMENT PLAN (ARCHITECTURE DIAGRAM)

After finishing all the planning and design work, I finally started the actual coding of the web application. But before writing the code, I needed a proper development plan to keep things organized. If I mixed the HTML designs, the PHP logic, and the database queries all in one place, the code would become very messy and hard to fix later.

To avoid this problem, I followed the Three-Tier Architecture model. This model divides the whole web application into three separate layers so that every part has its own specific job. It makes the system very easy to manage.

Here is how I divided the development of the "Access Form" system into three tiers:

4.1.1. Presentation Layer (Frontend):

This is the top layer that the user actually sees on their screen. I built this layer using HTML, CSS, and JavaScript. My main focus here was accessibility. The web browser handles the user's clicks, keyboard navigation, and runs the Web Speech API (for voice typing and screen reading).

4.1.2. Application Layer (Backend Logic):

This is the middle layer, basically the "brain" of the system. I wrote this layer using Core PHP. Its job is to take requests from the frontend, process them, and enforce the rules. For example, checking if the user typed the correct password, validating the branching logic of a survey, or generating a PDF report.

4.1.3. Data Layer (Database):

This is the bottom layer where all the information is permanently stored. I used MySQL for this. The Application layer (PHP) talks to this layer using secure PDO connections to fetch or save user records, accessibility settings, and survey answers.

By splitting my code into these three parts, the development process became very smooth. If I wanted to change the color-blind theme (Presentation Layer), it did not disturb my database queries (Data Layer).

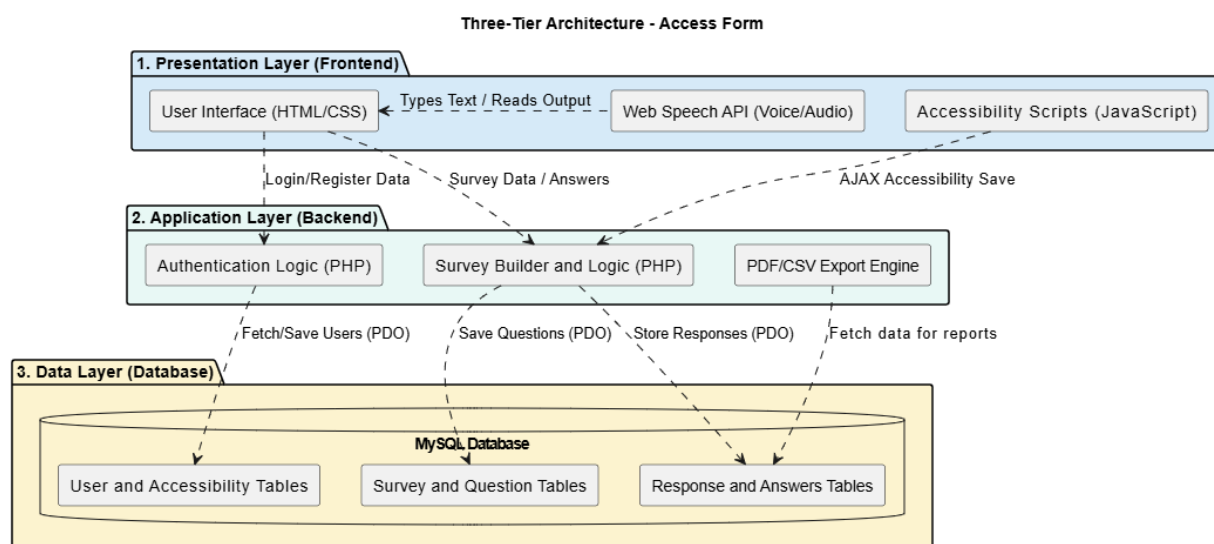


Figure 4.1 - 3-Tier Architecture Diagram

REFERENCES

During the research and development of the "Access Form" web application, I took help from several official documentations, articles, and libraries to ensure my code was secure and followed international accessibility standards. Below are the main references I used:

➤ **W3C Web Content Accessibility Guidelines (WCAG) 2.1:**

Official documentation by the World Wide Web Consortium to understand the rules of web accessibility, color contrast ratios, and keyboard navigation.

(Available at: <https://www.w3.org/TR/WCAG21/>)

➤ **MDN Web Docs – Web Speech API:**

Mozilla Developer Network's official guide. This was heavily used to learn how to integrate the Speech-to-Text (Voice Typing) and Text-to-Speech (Screen Reader) functionalities natively using JavaScript without third-party APIs.

(Available at: https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API)

➤ **PHP Official Documentation:**

Referred to for writing secure backend logic, managing user sessions, and implementing PHP Data Objects (PDO) to protect the MySQL database from SQL injection attacks.

(Available at: <https://www.php.net/manual/en/book.pdo.php>)

➤ **MySQL 8.0 Reference Manual:**

Used to understand relational database structures, designing the Entity Relationship Diagram (ERD), and writing efficient SQL queries to fetch survey and response data.

(Available at: <https://dev.mysql.com/doc/refman/8.0/en/>)

➤ **HTML2PDF.js Library (by eKoopmans):**

The open-source JavaScript library I integrated into the Admin and Creator panels. The documentation helped me convert HTML report tables into downloadable PDF files for offline use.

(Available at: <https://github.com/eKoopmans/html2pdf.js>)

➤ **WebAIM (Web Accessibility In Mind):**

An excellent resource that provided practical examples of how to design color-blind-friendly palettes and check contrast errors for visually impaired users.

(Available at: <https://webaim.org/>)

➤ **PlantUML Documentation:**

Used during the Design Phase (Chapter 3) to learn the syntax for generating professional UML diagrams, including Sequence Diagrams, Class Diagrams, and the Gantt Chart.

(Available at: <https://plantuml.com/>)

➤ **Apache Friends (XAMPP):**

The official documentation for setting up the local server environment. It helped in configuring the Apache server and PHPMyAdmin for local database management during the development phase.

(Available at: <https://www.apachefriends.org/>)

➤ **CSS-Tricks - A Complete Guide to Flexbox:**

A highly useful design reference that I used to structure the frontend layouts and make the dashboards responsive for different screen sizes.

(Available at: <https://css-tricks.com/snippets/css/a-guide-to-flexbox/>)

➤ **PHP Password Hashing Guidelines:**

Referred to the official PHP manual to understand the `password_hash()` and `password_verify()` functions. This was crucial for implementing strong security (bcrypt algorithm) for user passwords.

(Available at: <https://www.php.net/manual/en/faq.passwords.php>)

➤ **W3Schools Web Tutorials:**

Used as a quick reference guide for basic HTML5 form validations, CSS styling, and simple JavaScript event listeners during the frontend design phase.

(Available at: <https://www.w3schools.com/>)

➤ **Stack Overflow Developer Community:**

Used for troubleshooting and debugging complex coding issues, particularly when dealing with asynchronous JavaScript (AJAX) calls and dynamic DOM updates for the accessibility settings.

(Available at: <https://stackoverflow.com/>)

APPENDIX

This appendix contains two of the most critical and complex code snippets from the "Access Form" project. They demonstrate the core logical implementation of both the frontend accessibility features and the secure backend architecture.

1. Frontend Logic: Smart Voice Typing (JavaScript / Web Speech API)

The main focus of this project is accessibility. This JavaScript function allows physically impaired respondents to answer survey questions using their voice. It turns on the microphone, translates the speech to text, fills the correct input field, checks for branching logic, and gives audio feedback to the user.

```
// GLOBAL VOICE-TO-TEXT FUNCTION (accessibility.js)
function startSmartVoice(btnElement, questionId, type) {
  if (!window.SpeechRecognition && !window.webkitSpeechRecognition) return;
  var SpeechRecognition = window.SpeechRecognition ||
    window.webkitSpeechRecognition;
  var recognition = new SpeechRecognition();

  recognition.lang = "en-US";
  recognition.interimResults = false;

  // Visual recording animation and Audio Feedback
  btnElement.classList.add('recording');
  speakFeedback("Listening.");

  // Start listening after the screen reader stops talking
  let checkSpeaking = setInterval(() => {
    if (!window.speechSynthesis.speaking) {
      clearInterval(checkSpeaking);
      try { recognition.start(); } catch(err) {}
    }
  }, 200);

  // Process the voice result
  recognition.onresult = function(e) {
    var transcript = e.results[0][0].transcript.toLowerCase().replace(/[,?!]/g, "").trim();
    let matched = false;

    if (type === 'Multiple Choice') {
      let radios = Array.from(document.querySelectorAll('input[name="answers[' + questionId +
        ']]'));
      for (let radio of radios) {
        if (transcript.includes(radio.value.toLowerCase())) {
          radio.checked = true; matched = true;
          speakFeedback("Selected " + radio.value);
          evaluateBranching();
        }
      }
    }
  }
}
```

```

        break;
    }
}
    } else if (type === 'Text') {
        let textarea = document.getElementById('ans_' + questionId);
        textarea.value = (textarea.value + " " + transcript).trim();
        matched = true;
        speakFeedback("Text entered.");
        evaluateBranching();
    }

    if (!matched) speakFeedback("I heard '" + transcript + "', please try again.");
};
recognition.onend = function() { btnElement.classList.remove('recording'); };
}

```

2. Backend Logic: Security & Smart Auto-Configuration (Core PHP)

This backend PHP script handles new user registrations. It demonstrates strict password validation using Regular Expressions (Regex), secure password hashing to prevent database leaks, and a smart algorithm that automatically configures the system's settings (High Contrast, Large Font, Screen Reader) based on the user's selected disability profile.

```

// USER REGISTRATION & AUTO-CONFIGURATION (register_process.php)
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $name = trim($_POST['name']);
    $email = trim($_POST['email']);
    $password = $_POST['password'];
    $profile = isset($_POST['disability']) ? $_POST['disability'] : 'none';

    // 1. Backend Security: Strict Password Validation
    if (strlen($password) < 8 || !preg_match("/[A-Z]/", $password) || !preg_match("/[a-z]/", $password) || !preg_match("/\d/", $password) || !preg_match("/[\W_]/", $password)) {
        echo "<script>alert('Security Error: Password does not meet requirements.');"
        window.history.back();</script>";
        exit();
    }

    // Securely Hash the Password using bcrypt
    $hashed_password = password_hash($password, PASSWORD_DEFAULT);

    try {
        // 2. Insert User securely using PDO Prepared Statements
        $sql = "INSERT INTO user (name, email, password, role, disability_profile) VALUES (:name, :email, :password, 'Form Creator', :profile)";
        $stmt = $pdo->prepare($sql);
        $stmt->execute([':name' => $name, ':email' => $email, ':password' => $hashed_password, ':profile' => $profile]);
    }
}

```

```

        $new_user_id = $pdo->lastInsertId();

        // 3. Smart Auto-Configuration Algorithm
        $font_size = 'Normal'; $high_contrast = 0; $screen_reader = 0; $color_blind = 0;

        if ($profile === 'visual') {
            $font_size = 'Extra Large';
            $high_contrast = 1;
            $screen_reader = 1;
        } elseif ($profile === 'colorblind') {
            $color_blind = 1;
        } elseif ($profile === 'physical') {
            $screen_reader = 1;
        }

        // Insert auto-configured preferences into the database
        $sql_settings = "INSERT INTO accessibilitysettings (user_id, font_size, high_contrast,
            screen_reader, color_blind) VALUES (:uid, :font, :hc, :sr, :cb)";
        $stmt_settings = $pdo->prepare($sql_settings);
        $stmt_settings->execute([':uid' => $new_user_id, ':font' => $font_size, ':hc' =>
            $high_contrast, ':sr' => $screen_reader, ':cb' => $color_blind]);

        echo "<script>alert('Registration successful! Please login.');" window.location.href =
            'login.php';</script>";

        } catch (PDOException $e) {
            echo "<script>alert('Error: Email might already be registered.');"
                window.history.back();</script>";
        }
    }
}

```

3. Project Setup & Deployment Guide (Local Environment)

Unlike standard university projects that are only tested on local localhost environments, the "Access Form" system has been fully deployed on a live production server. This was necessary because modern browser accessibility features, specifically the Web Speech API (Microphone access for Voice Typing), strictly require a secure HTTPS connection to function.

Below are the steps I followed to deploy this project live on the internet:

➤ Step 1: Domain and Hosting Setup

I acquired a professional .me domain (accessform.me) using the GitHub Student Developer Pack. For the server, I utilized InfinityFree hosting to get a live Apache and PHP environment.

➤ Step 2: Database Migration to Live Server

Through the InfinityFree Control Panel (cPanel), I accessed the live phpMyAdmin. I created a new MySQL database (e.g.,

if0_41714280_db_accessform) and imported my local db_accessform.sql file to create all the necessary tables (User, Survey, Question, Response, etc.) on the live server.

➤ **Step 3: Updating Configuration Files**

Before uploading the code, I updated the includes/db_connection.php file to match the live server's credentials. I replaced the local root credentials with the live InfinityFree database username, password, and hostname.

➤ **Step 4: Uploading Files via FTP**

Using the online File Manager (and FTP client), I uploaded the entire project directory into the live htdocs folder. I ensured that all modular folders (admin, creator, respondent, assets) were correctly structured.

➤ **Step 5: Enforcing SSL/HTTPS**

To ensure the Screen Reader and Voice Typing features work without browser security blocks, I configured an SSL certificate for the domain. I also added an .htaccess file to automatically redirect all HTTP traffic to secure HTTPS.

➤ **Step 6: Live Access**

The system is now fully live and globally accessible. Anyone can open a modern web browser and visit <https://www.accessform.me> to register, create surveys, or test the accessibility features in real-time.